

Arquitectura de Datos

Especialización en Internet de las Cosas

Docente:

Lacheski, Martín Anibal

Clase 1:

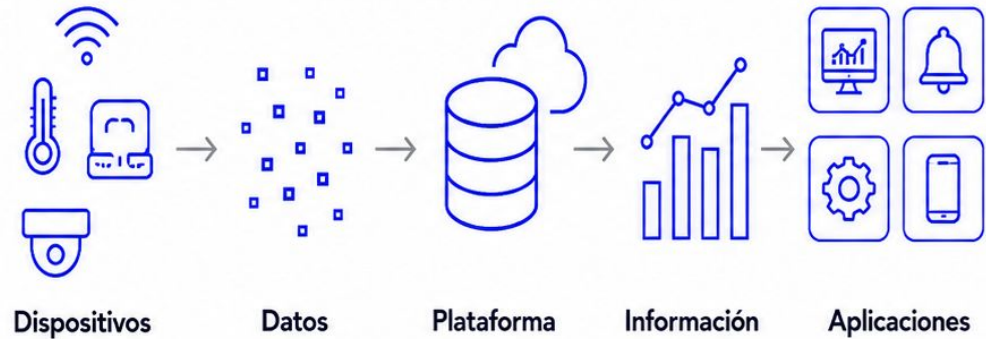
Introducción a la Arquitectura de Datos



Arquitectura de Datos

¿Por qué una materia de datos en Internet de las Cosas?

- Los sistemas IoT generan información de manera continua.
- Los datos provienen de dispositivos, sensores, aplicaciones y servicios externos.
- Deben almacenarse, consultarse, procesarse y mantenerse disponibles.
- Diferentes datos requieren diferentes mecanismos de persistencia.
- El crecimiento del sistema modifica los requerimientos de almacenamiento.



Conectar dispositivos permite generar datos.
La arquitectura determina qué podemos hacer con ellos.

Del dispositivo a la información

Capturar un dato es solamente el primer paso

Una solución IoT debe responder preguntas como:

- ¿qué información genera cada dispositivo?;
- ¿cómo ingresa al sistema?;
- ¿dónde se almacena?;
- ¿durante cuánto tiempo?;
- ¿cómo se consulta?;
- ¿quién puede acceder?;
- ¿qué ocurre cuando aumenta el volumen?



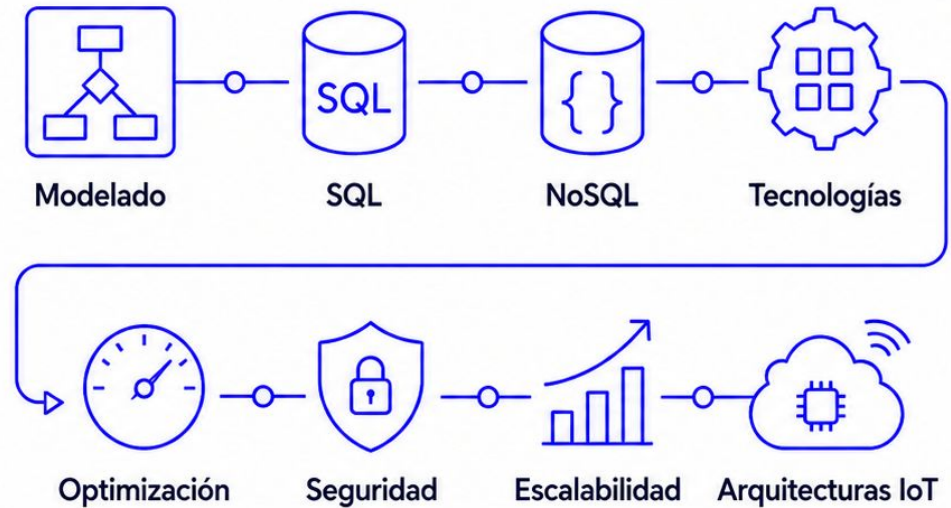
Cada dato tiene un recorrido dentro del sistema.

Arquitectura de Datos

¿Qué vamos a aprender?

Al finalizar la materia deberíamos poder:

- diseñar modelos de datos;
- utilizar bases relacionales y SQL;
- reconocer diferentes modelos NoSQL;
- seleccionar tecnologías especializadas;
- administrar y optimizar bases de datos;
- comprender mecanismos de seguridad y recuperación;
- analizar estrategias de escalabilidad;
- diseñar arquitecturas adecuadas para diferentes escenarios IoT.



**El objetivo no es aprender productos.
Es desarrollar criterio para seleccionar y combinar tecnologías.**

Arquitectura de Datos

Recorrido de la materia

- Clase 1 → Introducción a la Arquitectura de Datos
- Clase 2 → Modelado de Datos
- Clase 3 → Bases de Datos Relacionales y SQL
- Clase 4 → Diseño Relacional y Optimización
- Clase 5 → Bases de Datos NoSQL
- Clase 6 → Bases Especializadas para IoT
- Clase 7 → Administración, Seguridad y Escalabilidad
- Clase 8 → Arquitecturas de Datos y Grandes Volúmenes



El objetivo no es aprender productos.

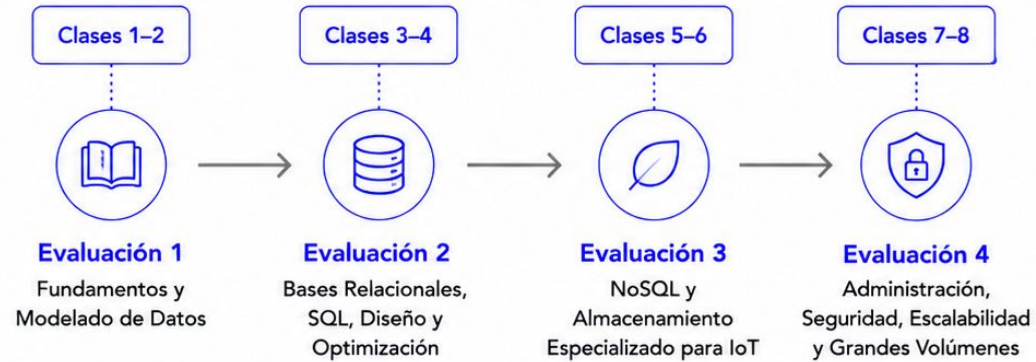
Es desarrollar criterio para seleccionar y combinar tecnologías.

Arquitectura de Datos

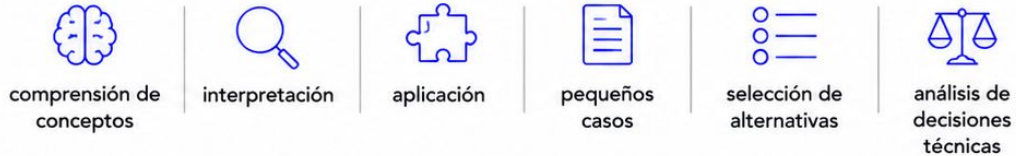
Metodología de evaluación

Durante la materia tendremos 4 evaluaciones en Moodle, organizadas por bloques de contenidos:

-  **Evaluación 1**
Fundamentos y Modelado de Datos
-  **Evaluación 2**
Bases Relacionales, SQL, Diseño y Optimización
-  **Evaluación 3**
NoSQL y Almacenamiento Especializado para IoT
-  **Evaluación 4**
Administración, Seguridad, Escalabilidad y Grandes Volúmenes



Las evaluaciones combinarán:



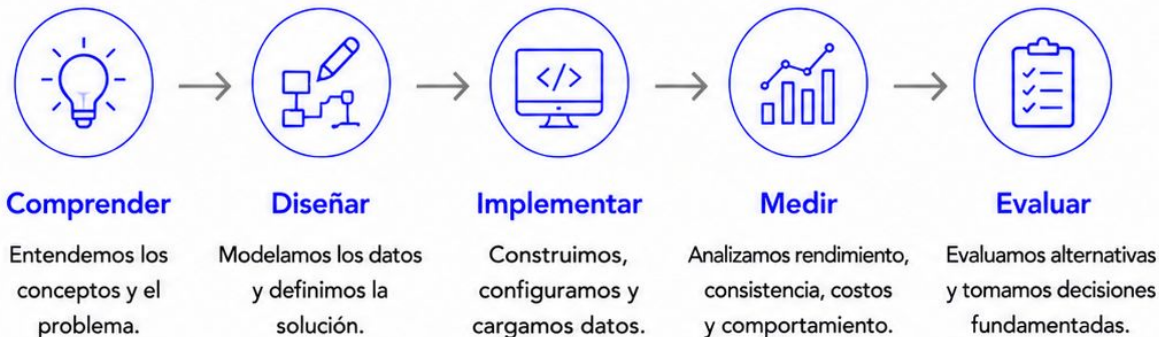
La evaluación acompañará el desarrollo de la materia, con una instancia al finalizar cada bloque de contenidos.

Modalidad de trabajo

Conceptos, implementación y análisis

Durante la materia vamos a combinar:

- fundamentos conceptuales;
- análisis de casos;
- modelado de datos;
- consultas SQL;
- experimentación con diferentes tecnologías;
- administración y optimización;
- diseño y comparación de arquitecturas.



Implementar es importante.

Entender por qué elegimos una solución lo es todavía más.

Arquitectura de Datos

Agenda del día

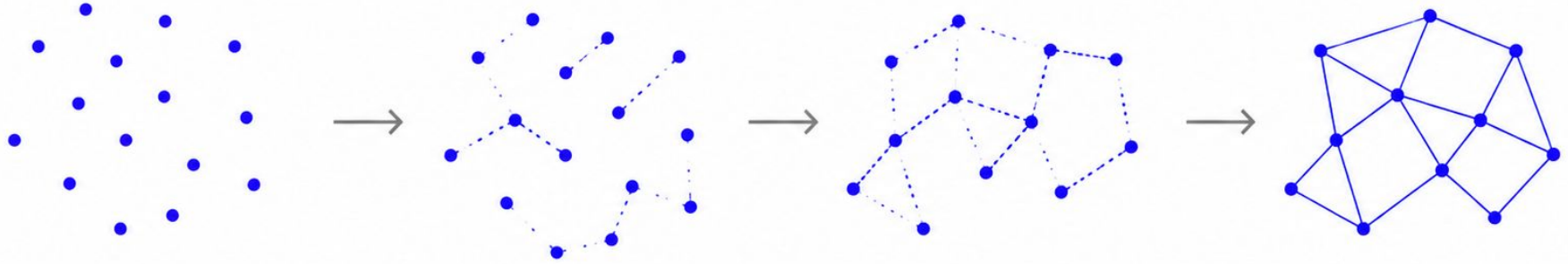
- Dato, información y conocimiento.
- Arquitectura de datos.
- Arquitectura de datos dentro de IoT.
- Fuentes, ingreso, almacenamiento, procesamiento y consumo.
- Evolución de las tecnologías de almacenamiento.
- Bases de datos y sistemas gestores.
- Datos estructurados, semiestructurados y no estructurados.
- Clasificación de bases de datos y almacenamiento.
- Persistencia y ciclo de vida.
- Panorama de una arquitectura IoT.



Esta clase sienta las bases para comprender
cómo los datos se convierten en un activo estratégico en IoT.

1

Dato, Información y Conocimiento



Comprender el recorrido transforma datos en decisiones y valor.

¿Qué es un dato?

El punto de partida

- Representa una observación, medición, evento o característica.
- Puede provenir de una persona, aplicación o dispositivo.
- Aislado puede no tener suficiente significado.

?

?

24.6

?

?



24.6 no significa demasiado si no sabemos qué representa.

¿Qué es información?

Cuando el dato adquiere significado

24.6

Dato



24.6 °C

Dato con unidad



	24.6 °C
	Sensor A01
	Aula 204
	10:32:15

Información



El contexto transforma un valor aislado en información interpretable.

¿Qué es conocimiento?

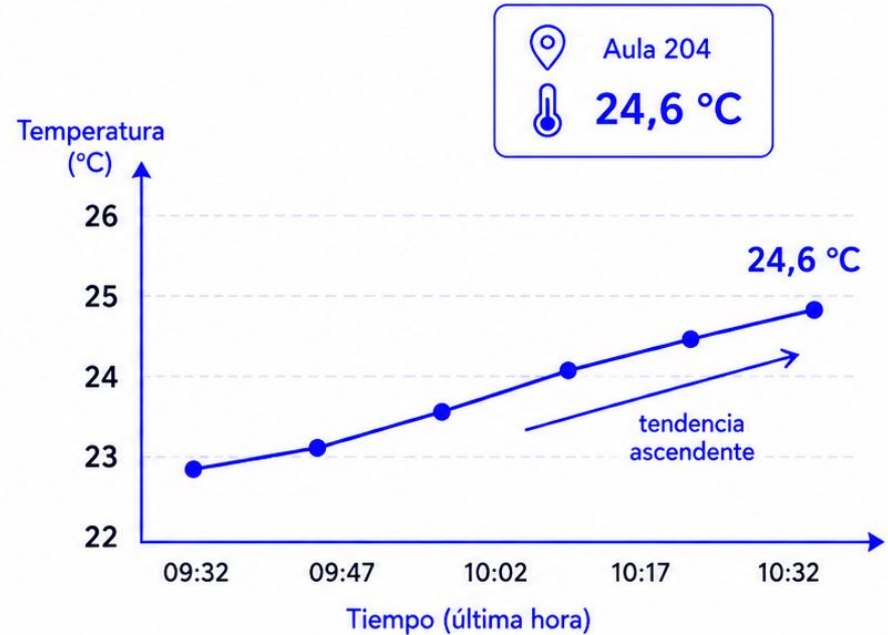
Encontrar patrones para poder actuar

Información:

La temperatura actual del Aula 204 es 24,6 °C.

Conocimiento:

La temperatura del ambiente aumentó durante la última hora y presenta una tendencia ascendente.



El conocimiento permite utilizar información para tomar decisiones.

Pirámide DIKW

Datos, información, conocimiento y decisión

Datos

24.6

Información

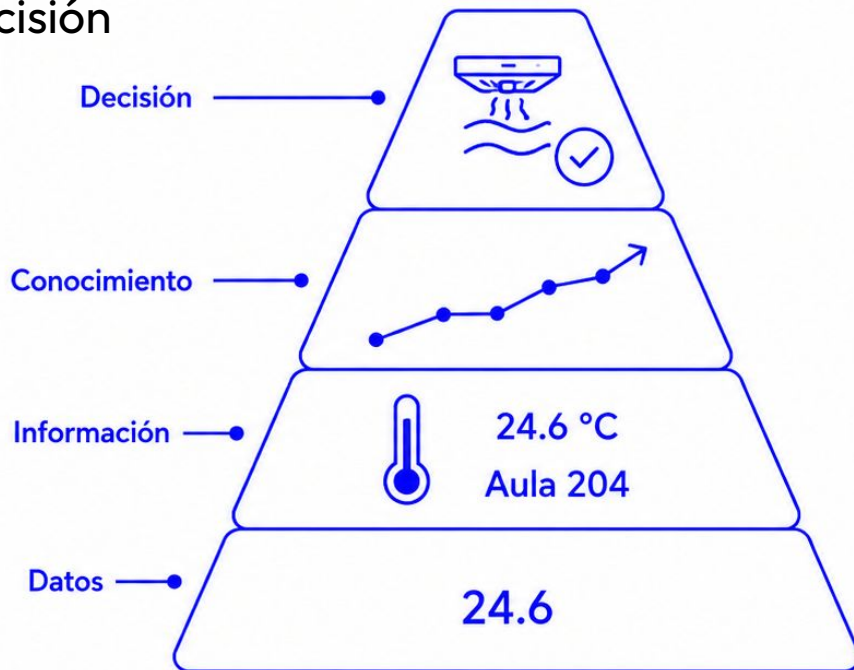
24.6 °C en el Aula 204.

Conocimiento

La temperatura viene aumentando.

Decisión

Revisar las condiciones ambientales o la ventilación.



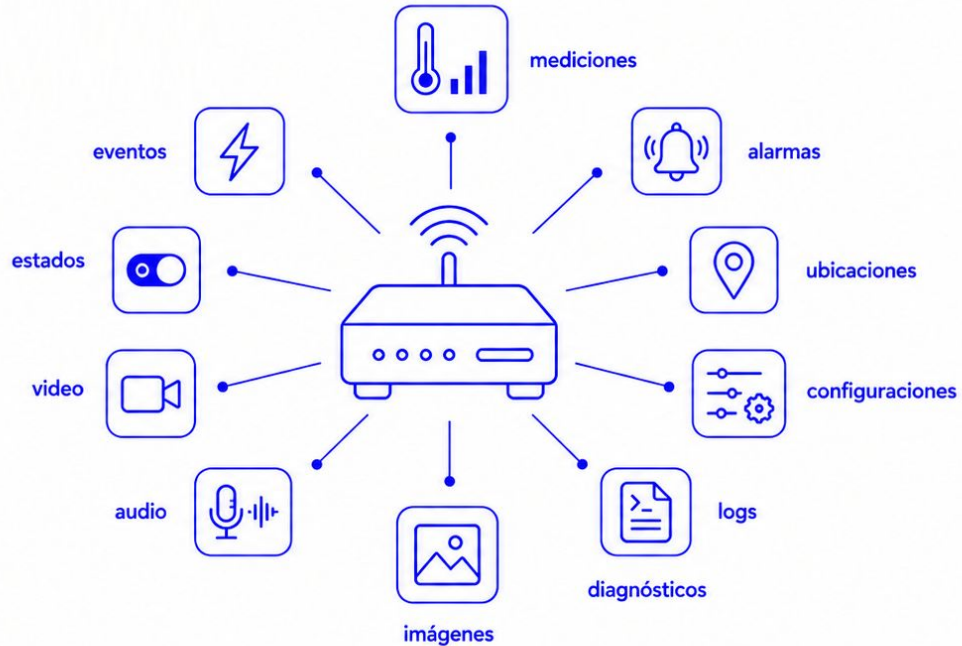
Cada nivel agrega contexto, interpretación y capacidad de decisión.

¿Qué genera un sistema IoT?

Mucho más que mediciones

Los sistemas IoT pueden producir:

- mediciones;
- eventos;
- estados;
- alarmas;
- ubicaciones;
- configuraciones;
- logs;
- diagnósticos;
- imágenes;
- audio;
- video.



Un sistema IoT puede combinar datos muy diferentes dentro de una misma solución.

El valor no es suficiente

¿Qué necesitamos saber además de la medición?

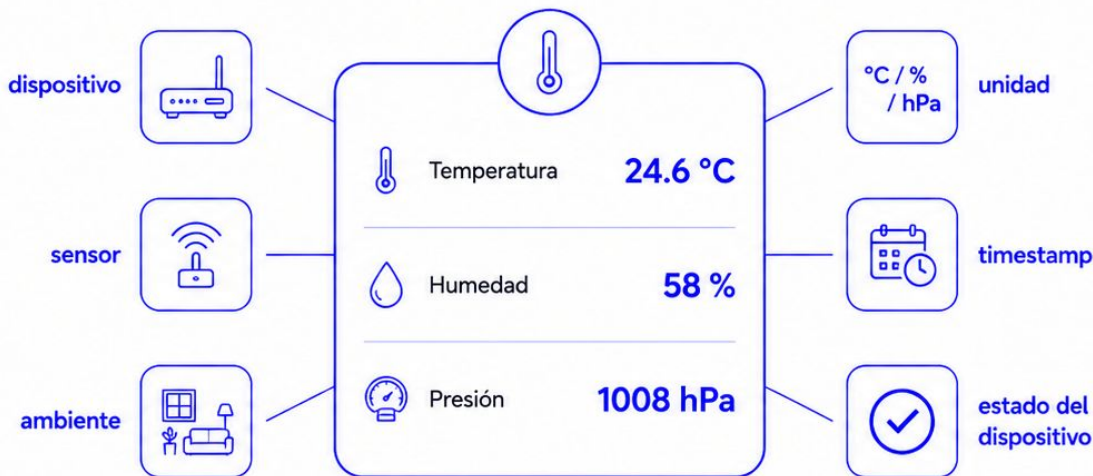
Temperatura: **24.6 °C**

Humedad: **58 %**

Presión: **1008 hPa**

Además podemos necesitar:

- dispositivo;
- sensor;
- ambiente;
- unidad;
- timestamp;
- estado del dispositivo.

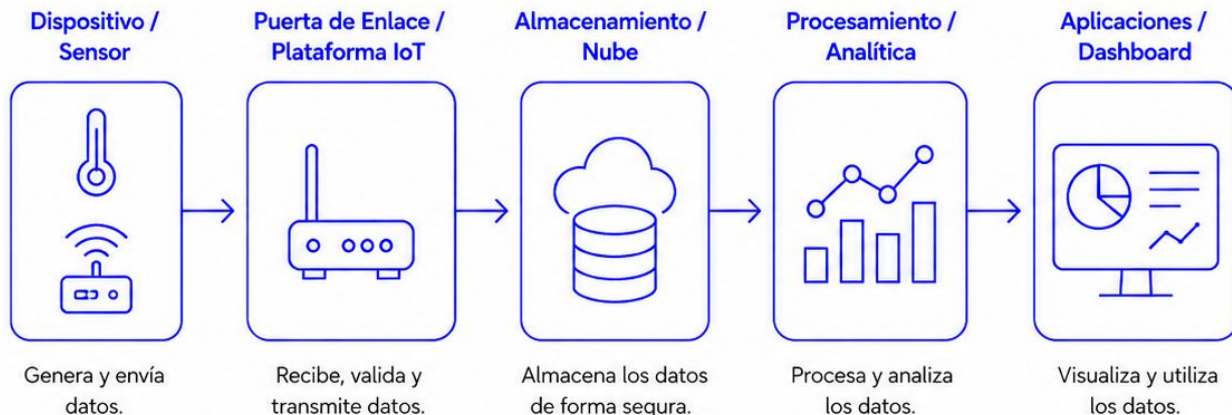


El dato IoT normalmente necesita contexto temporal y operacional.

2

Arquitectura de Datos

Del dato generado al dato utilizado.



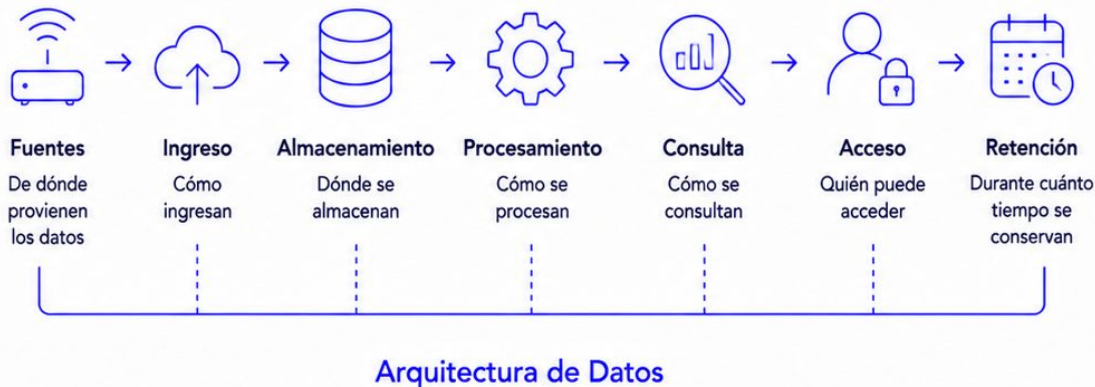
Del dato generado al dato utilizado.

Arquitectura de Datos

Organizar cómo vive la información dentro del sistema

Una arquitectura de datos define:

- de dónde provienen los datos;
- cómo ingresan;
- dónde se almacenan;
- cómo se procesan;
- cómo se consultan;
- quién puede acceder;
- durante cuánto tiempo se conservan.



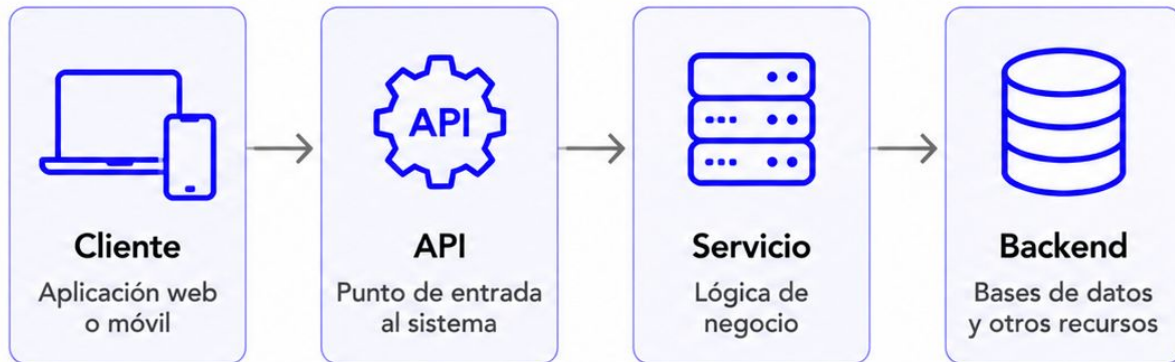
Una base de datos es un componente de la arquitectura de datos.
No es toda la arquitectura.

Arquitectura de Datos

Organizar componentes y comportamiento

La arquitectura de software describe componentes como:

- aplicaciones;
- servicios;
- APIs;
- procesos;
- interfaces;
- comunicaciones.



La arquitectura de software define cómo se integran y coordinan los componentes del sistema.

Arquitectura de Datos

Organizar información y persistencia

La arquitectura de datos se concentra en:

- fuentes;
- estructuras;
- almacenamiento;
- transformación;
- acceso;
- movimiento;
- persistencia;
- ciclo de vida.



Software y datos representan dos perspectivas del mismo sistema.

Dos perspectivas

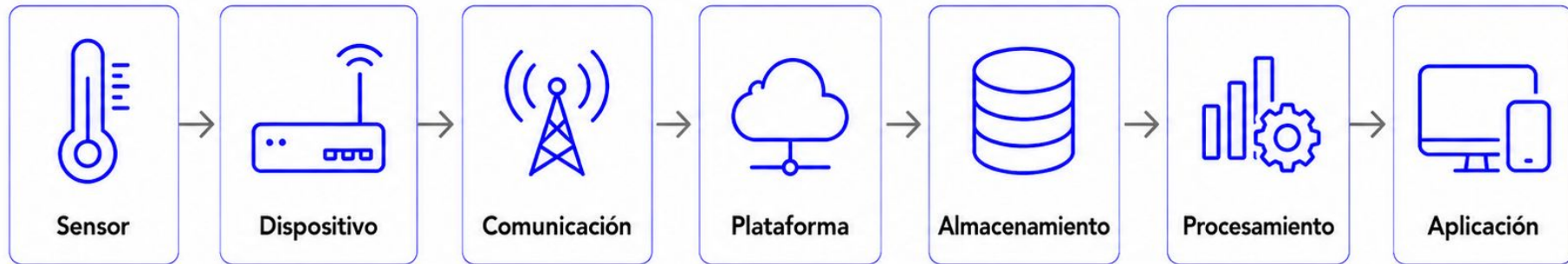
Arquitectura de Software + Arquitectura de Datos



Diseñar software sin pensar en los datos genera decisiones incompletas.

El recorrido de una medición

El dato atraviesa diferentes componentes



Una medición puede atravesar múltiples sistemas antes de llegar al usuario.

Flujo de datos

Cinco etapas fundamentales



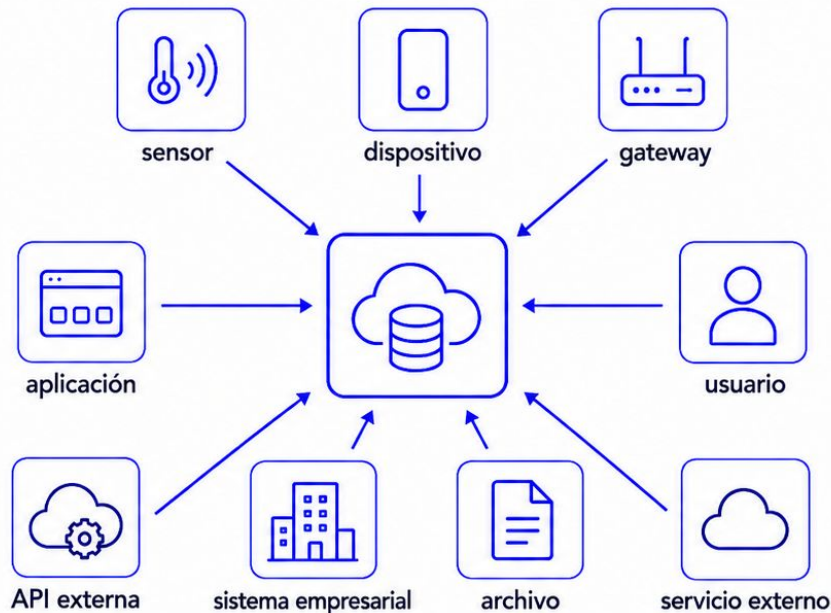
Una arquitectura de datos puede estudiarse siguiendo el recorrido de la información.

Fuentes de datos

¿Dónde nace la información?

En una solución IoT puede provenir de:

- sensores;
- dispositivos;
- gateways;
- aplicaciones;
- usuarios;
- APIs externas;
- sistemas empresariales;
- archivos;
- servicios externos.



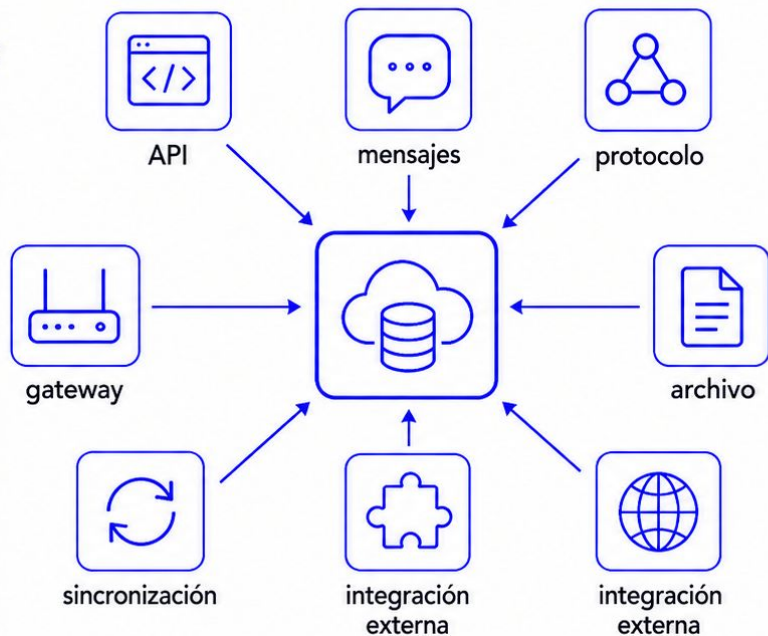
No todos los datos de un sistema IoT provienen directamente de sensores.

Ingreso de datos

¿Cómo llega la información al sistema?

Los datos pueden ingresar mediante:

- APIs;
- mensajes;
- protocolos de comunicación;
- gateways;
- archivos;
- sincronizaciones;
- integraciones externas.



Transportar un dato y almacenarlo son problemas diferentes.

Almacenamiento

¿Dónde permanece la información?

Una solución puede necesitar almacenar:

- dispositivos;
- usuarios;
- configuraciones;
- mediciones;
- eventos;
- estados;
- archivos;
- información histórica.



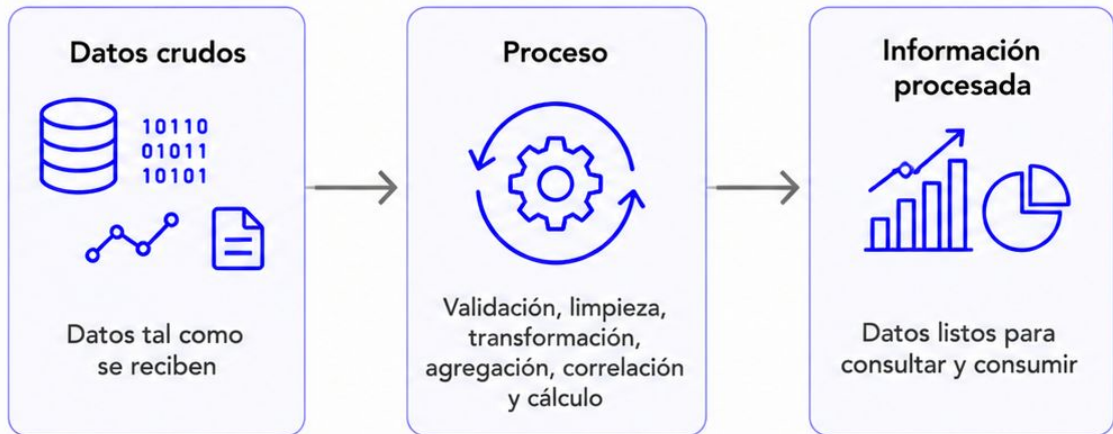
Diferentes tipos de información pueden requerir diferentes formas de persistencia.

Procesamiento

Transformar datos para utilizarlos

Los datos pueden requerir:

- validación;
- limpieza;
- transformación;
- agregación;
- correlación;
- cálculo de indicadores;
- detección de eventos.



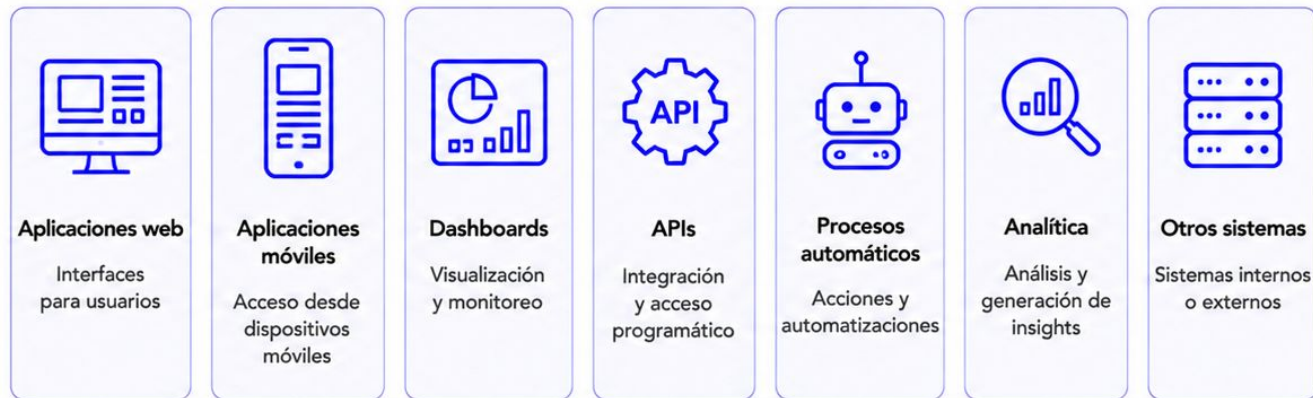
Diferentes tipos de información pueden requerir diferentes formas de persistencia.

Acceso y consumo

¿Quién necesita los datos?

La información puede ser utilizada por:

- aplicaciones web;
- aplicaciones móviles;
- dashboards;
- APIs;
- procesos automáticos;
- analítica;
- otros sistemas.



La arquitectura debe considerar cómo serán utilizados los datos, no solamente cómo serán almacenados.

Dos preguntas diferentes

¿Qué ocurre ahora y qué ocurrió antes?



Estado actual

¿Cuáles son la temperatura, humedad y presión actuales del Aula 204?

Estado actual – Aula 204



Temperatura
23.6 °C



Humedad
56.2 %



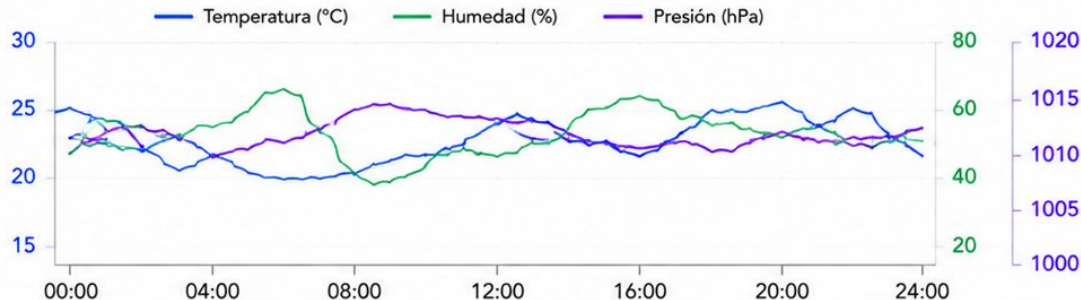
Presión
1012.3 hPa



Histórico

¿Cómo evolucionaron esas variables durante las últimas 24 horas?

Histórico – Últimas 24 horas

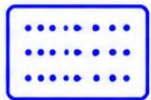


El mismo dato puede requerir diferentes formas de acceso según la pregunta.

3

Evolución del Almacenamiento de Datos

Cada salto tecnológico respondió a nuevos problemas.



1960s

Archivos y
procesamiento
por lotes



1970s–1980s

Bases de datos
relacionales



1990s

Data Warehouses
y OLAP



2000s–2010s

Big Data y
almacenamiento
distribuido



2010s–2020s

Nuevos modelos:
NoSQL, streaming,
lakehouse



2020s+

Datos para IA,
tiempo real y a
gran escala



Cada salto tecnológico respondió a nuevos problemas.

Evolución de las tecnologías

De archivos a arquitecturas distribuidas



Archivos y procesamiento secuencial



Modelos jerárquicos y en red



Modelo relacional



Bases distribuidas y sistemas analíticos



NoSQL



Tecnologías especializadas



Arquitecturas modernas de datos



Las tecnologías no siempre se reemplazan.
Se acumulan para resolver problemas diferentes.

Antes de las bases de datos

Los datos también podían almacenarse



Archivos:

cada aplicación guardaba sus datos en sus propios archivos.



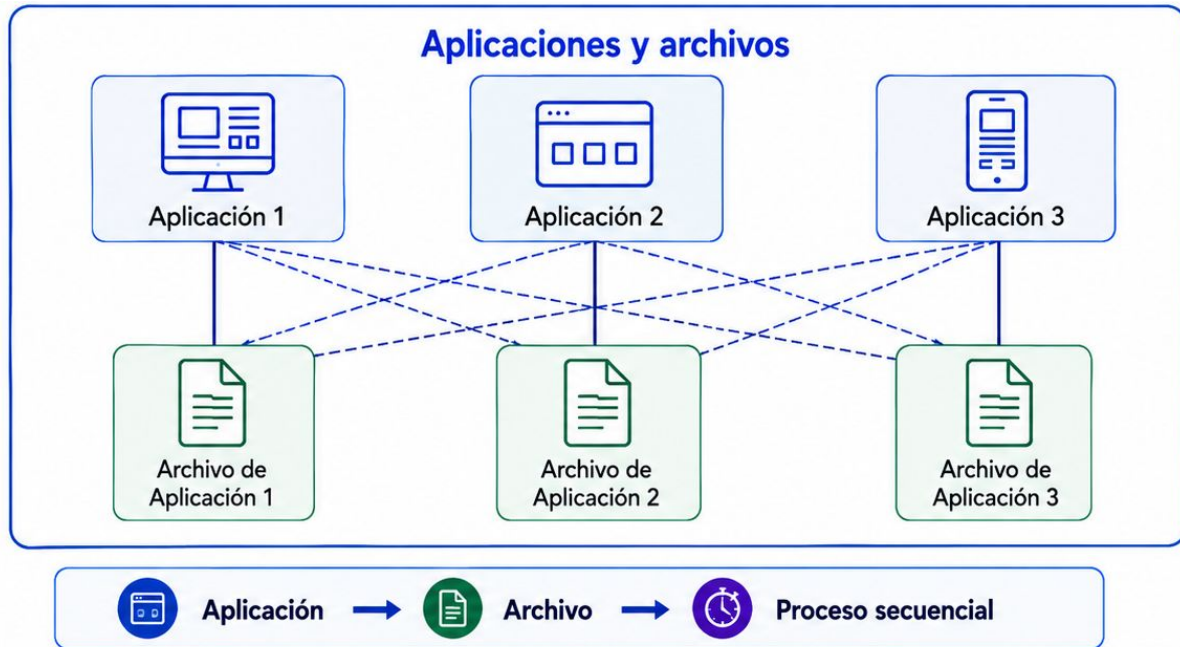
Formatos:

los formatos estaban definidos por cada aplicación.



Procesamiento secuencial

y estructuras acopladas al software.



**Guardar información no era el problema principal.
Administrarla era mucho más difícil.**

Primeros modelos de BD

1960s-1970s



El modelo jerárquico organizaba datos como árboles.

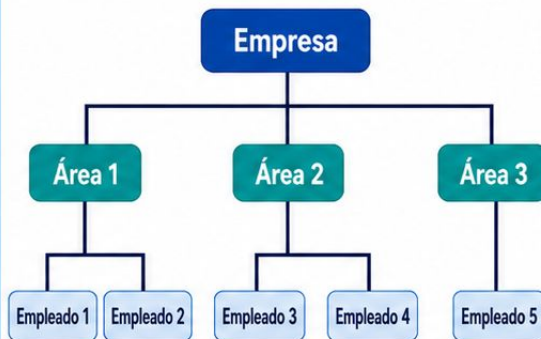


El modelo en red permitía múltiples relaciones.



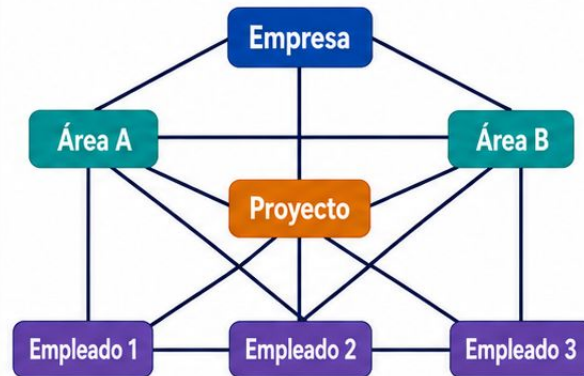
Eran eficientes, pero fuertemente dependientes de su estructura.

1 Jerárquico



Relaciones de padre-hijo,
una ruta principal.

2 En red



Un registro puede relacionarse
con varios otros.

Empresa → Área → Empleado



Modificar la estructura podía implicar modificar también las aplicaciones.

Revolución Relacional

1970s–1980s



Los datos se representan mediante relaciones.



Las tablas ofrecen una abstracción simple.



Las consultas se separan del almacenamiento físico.



SQL permite expresar qué información queremos obtener.

1 Modelo relacional

Áreas	
id_area	nombre
1	Finanzas
2	Recursos Humanos
3	Tecnología

PK
id_area

FK
id_area

Empleados		
id_empleado	nombre	id_area
101	Ana López	1
102	Carlos Díaz	1
103	María Gómez	2
104	Luis Pérez	3
105	Javier Ruiz	3

2 Consulta SQL

```
SELECT e.nombre, a.nombre  
FROM Empleados e  
JOIN Áreas a ON e.id_area = a.id_area;
```

3

La consulta pregunta por qué datos queremos, no por cómo se almacenan físicamente.



Qué queremos consultar $\neq$ cómo está físicamente almacenado.

Nuevos requerimientos

1990s–2000s

Nuevas necesidades, nuevas bases



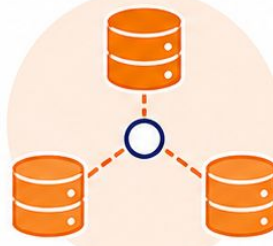
**Bases
Orientadas a Objetos**

Almacenan objetos
directamente



**Bases
Objeto-Relacionales**

SQL + tipos complejos
y herencia



**Bases
Distribuidas**

Datos en distintos nodos



Data Warehouse

Análisis y toma
de decisiones



**Nuevas necesidades exigieron nuevos modelos,
arquitecturas y sistemas de análisis.**

NoSQL

2000s–2010s

Internet introdujo nuevos desafíos:

- millones de usuarios;
- grandes volúmenes;
- datos variables;
- sistemas distribuidos;
- crecimiento horizontal.

Aparecieron diferentes modelos de almacenamiento NoSQL.

Modelos NoSQL



Documentales

Datos como documentos (JSON, XML)

(ej. MongoDB)



Clave-Valor

Pares clave-valor simples

(ej. Redis)



Columnas

Datos organizados por columnas

(ej. Cassandra, HBase)



Grafos

Nodos y relaciones entre ellos

(ej. Neo4j)



Búsqueda

Búsqueda y análisis de texto

(ej. Elasticsearch)



NoSQL amplió las alternativas disponibles para diseñar sistemas de datos.

Bases especializadas

Cuando un problema domina el diseño

Se consolidaron tecnologías optimizadas para:

- series temporales;
- grafos;
- datos geográficos;
- búsqueda;
- memoria;
- análisis;
- grandes archivos.



Series temporales

Datos medidos en el tiempo y ordenados cronológicamente.



Grafos

Entidades conectadas a través de relaciones complejas.



Datos geográficos

Información asociada a ubicaciones en el espacio.



Búsqueda

Indexación y búsqueda rápida de texto o documentos.



Memoria

Acceso ultra rápido para datos volátiles o de sesión.



Análisis

Consultas analíticas complejas sobre grandes volúmenes.



Grandes archivos

Almacenamiento eficiente de archivos extensos y no estructurados.



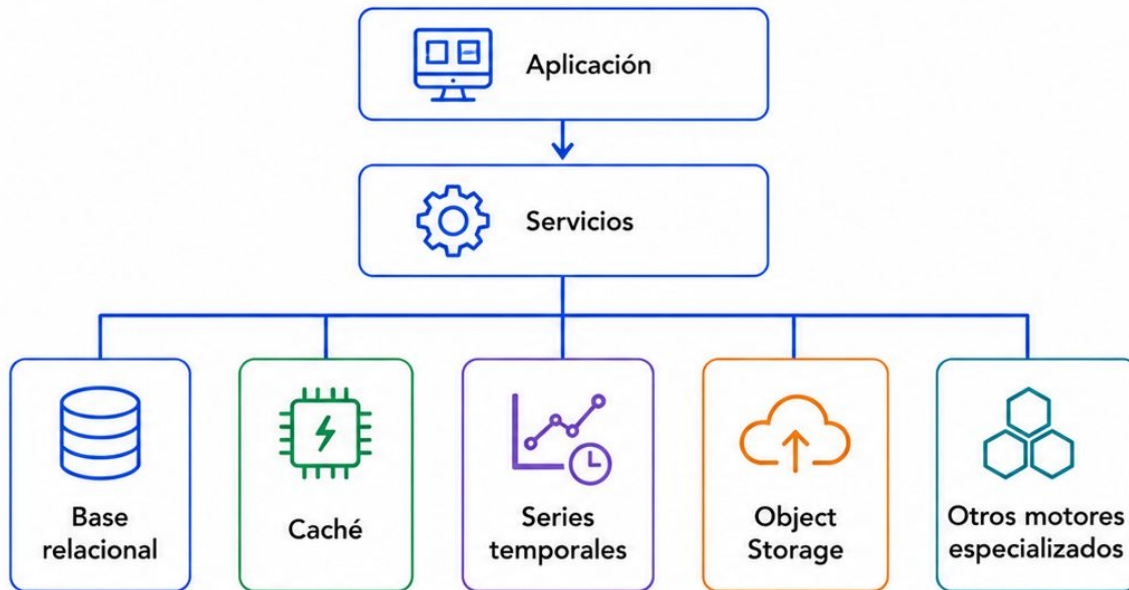
Un motor especializado puede resolver muy bien un problema específico.

La arquitectura actual

Una solución puede utilizar múltiples tecnologías

Una solución moderna combina componentes con propósitos diferentes:

- Base relacional
- Caché
- Series temporales
- Object Storage
- Otros motores especializados



**La pregunta ya no siempre es qué base de datos utilizar.
También puede ser qué combinación utilizar.**

4

Bases de Datos y Sistemas Gestores



Organizar, consultar y proteger información.

Base de Datos

Información organizada y persistente

Una base de datos permite mantener información:



estructurada;



relacionada;



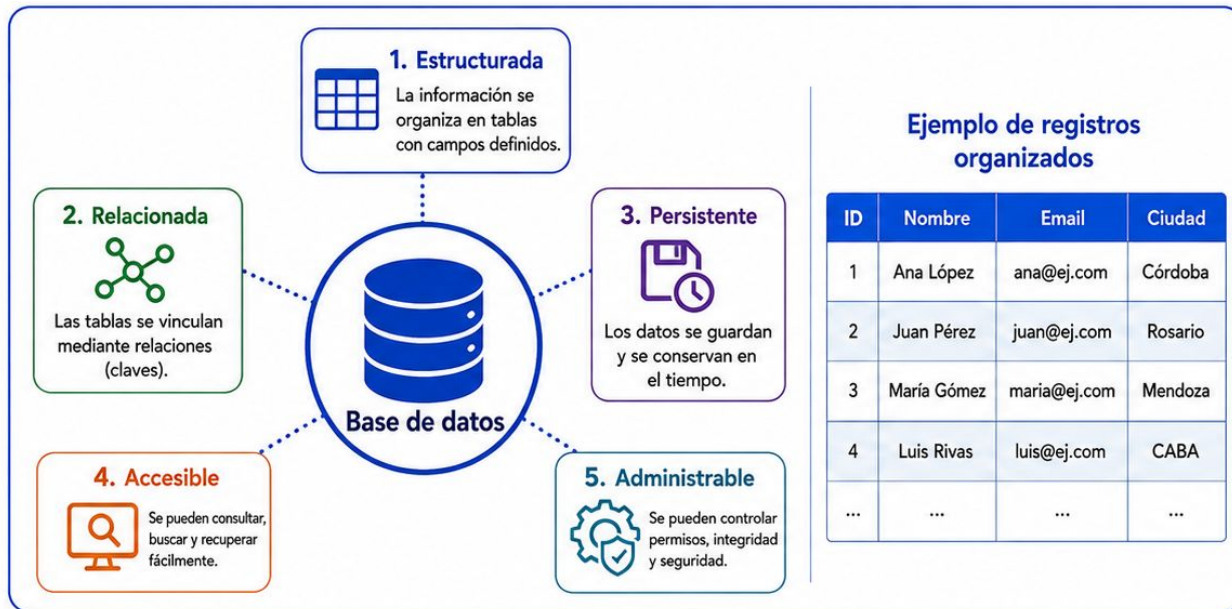
persistente;



accesible;



administrable.



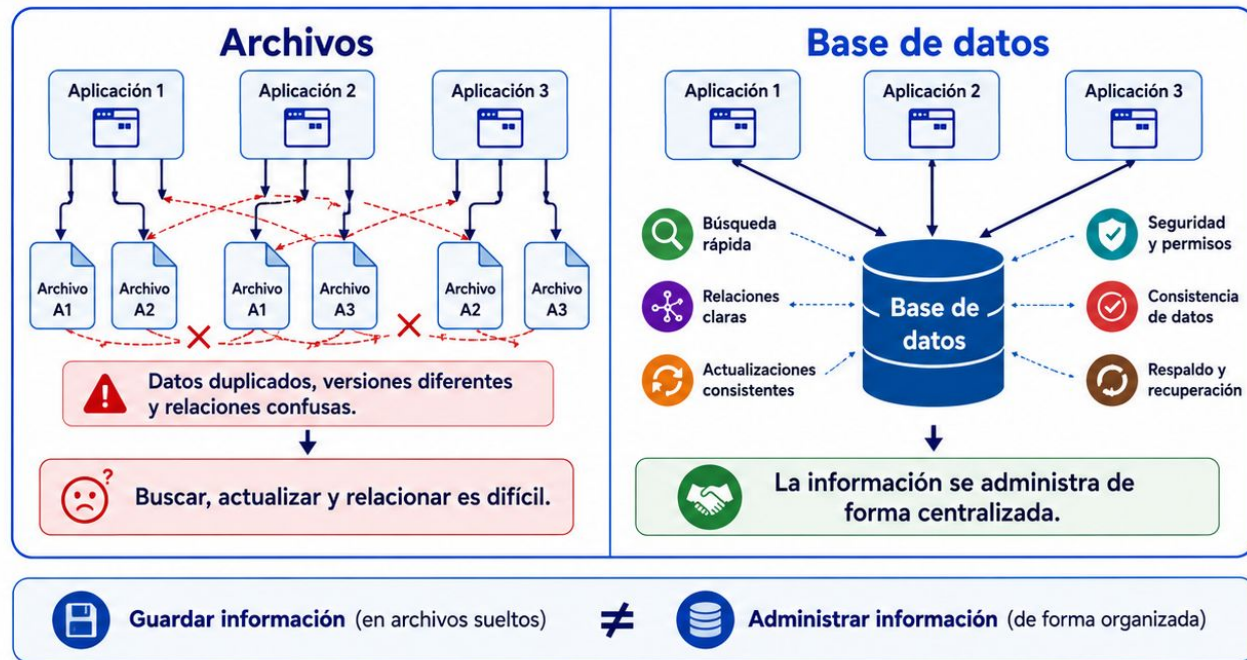
**No alcanza con tener datos.
Necesitamos poder organizarlos y recuperarlos.**

¿Por qué no utilizar archivos?

Almacenar no significa administrar

Cuando crece un sistema aparecen problemas de:

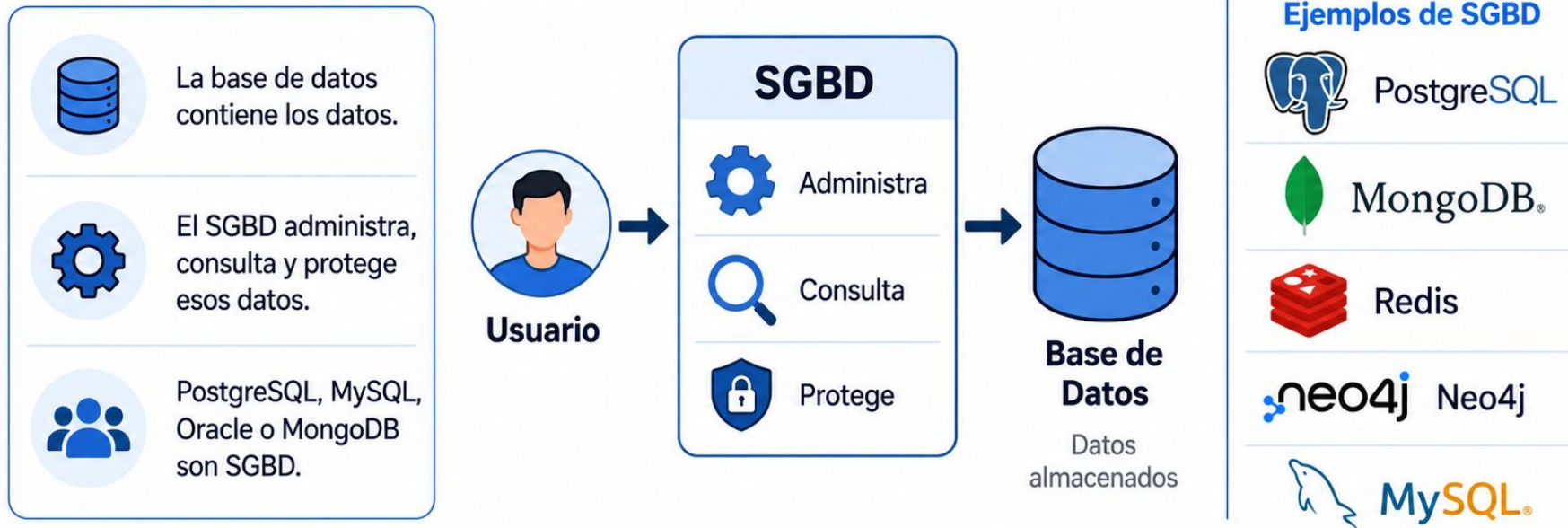
-  búsqueda;
-  duplicación;
-  relaciones;
-  actualizaciones;
-  concurrencia;
-  consistencia;
-  permisos;
-  recuperación.



**Un archivo puede guardar información.
Una base de datos ayuda a administrarla.**

¿Qué es un SGBD?

El software que administra la base



Base de Datos = contenido



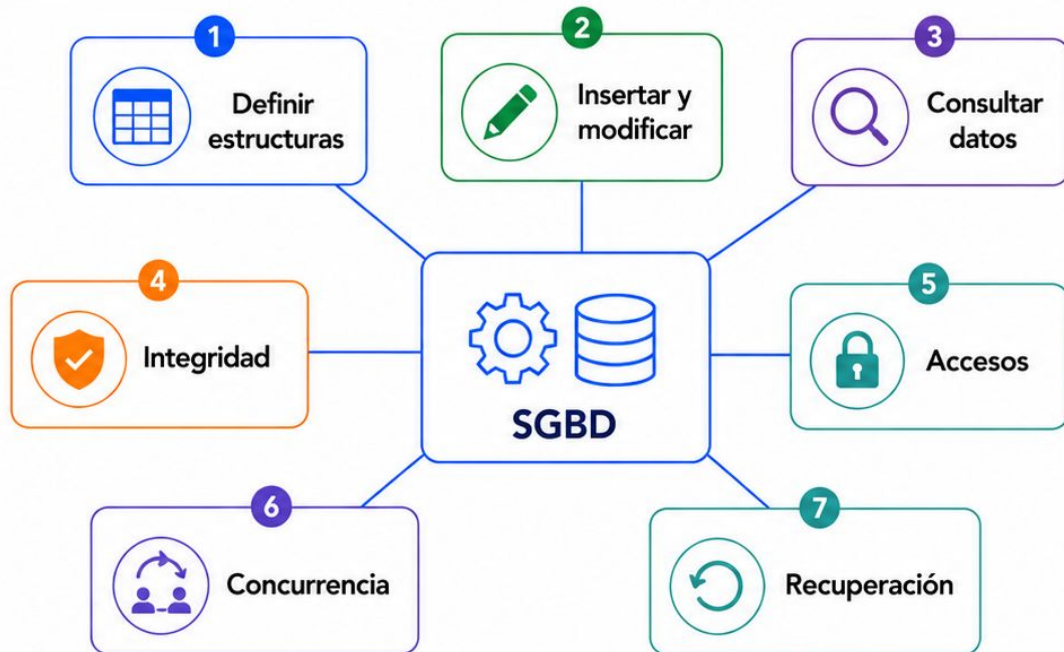
SGBD = software que lo administra

Funciones de un SGBD

Mucho más que guardar datos

Un SGBD permite:

- definir estructuras;
- insertar y modificar información;
- consultar datos;
- mantener reglas de integridad;
- gestionar accesos;
- coordinar operaciones simultáneas;
- recuperar información ante fallos.



**Un SGBD no solo guarda datos.
También los organiza, protege y recupera.**

Dentro del SGBD

Una visión simplificada

El flujo básico de una consulta:



También intervienen:



- memoria;



- seguridad;



- transacciones;



- registros;



- recuperación.



Una consulta simple puede activar muchos mecanismos internos.

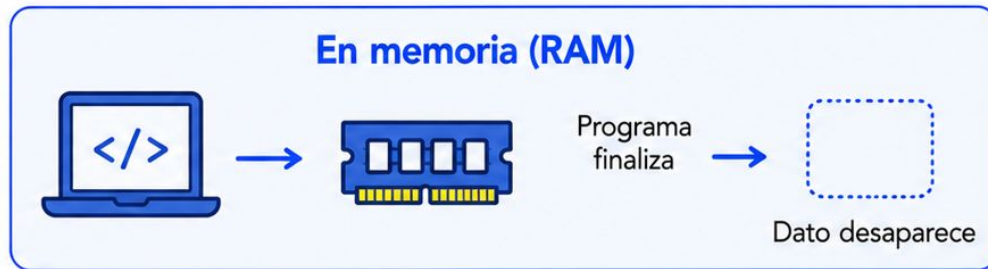
Persistencia

Conservar información después de ejecutar una aplicación



En memoria:

Programa finaliza →
dato desaparece.



Persistido:

Programa finaliza →
dato permanece disponible.



Persistir significa conservar información para poder recuperarla posteriormente.

5

Tipos de Datos

Estructurados, semiestructurados y no estructurados.

id	nombre	edad
1	Ana	28
2	Luis	35
3	María	22

```
{  
  "id": 1,  
  "nombre": "Ana",  
  "edad": 28,  
  "activo": true  
}
```



Cada tipo de dato requiere diferentes formas de almacenamiento y consulta.

Clasificación por estructura

¿Cuánto conocemos del dato antes de almacenarlo?



- Datos estructurados.

id	nombre	edad	ciudad
1	Ana	28	Posadas
2	Luis	35	Oberá
3	María	22	Eldorado



- Datos semiestructurados.

```
{  
  "id": 1,  
  "nombre": "Ana",  
  "edad": 28,  
  "ciudad": "Posadas",  
  "telefonos": ["3764-111111", "3764-222222"]  
}
```



- Datos no estructurados.



La estructura condiciona cómo almacenamos, consultamos y procesamos la información.

Datos estructurados

Cuando conocemos el esquema



Características:

- campos definidos;
- tipos conocidos;
- reglas claras;
- relaciones identificables;
- consultas precisas.

Ejemplo: mediciones ambientales

dispositivo	timestamp	temperatura	humedad	presión
A01	10:32	24.6	58	1008
A02	10:32	23.9	61	1007



Cuando conocemos el esquema, podemos validar, relacionar y consultar con precisión.

Datos semiestructurados

Existe estructura, pero puede variar



Son frecuentes en:

- APIs;
- mensajes;
- eventos;
- configuraciones;
- telemetría;
- logs.



Ejemplo

```
{  
  "device_id": "ambiente-01",  
  "temperature": 24.6,  
  "humidity": 58,  
  "pressure": 1008  
}
```



Los datos poseen organización, pero no necesariamente una estructura rígida.

Dispositivos diferentes

No todos generan exactamente los mismos atributos

Cada dispositivo puede:

- incluir atributos distintos;
- omitir algunos campos;
- agregar información nueva;
- cambiar el orden;
- variar los tipos de datos.

Dispositivo A

```
{  
  "temperature": 24.6,  
  "humidity": 58  
}
```

Dispositivo B

```
{  
  "temperature": 24.6,  
  "humidity": 58,  
  "pressure": 1008  
}
```



La heterogeneidad de dispositivos genera estructuras de datos variables.

Datos no estructurados

Cuando no existe una estructura explícita de filas y columnas



Ejemplos en IoT:

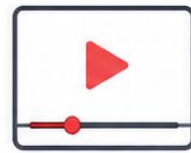
- imágenes;
- audio;
- video;
- documentos;
- archivos de diagnóstico;
- registros binarios.



Imagen



Audio



Video



Documento



Diagnóstico



Binario



El contenido tiene información aunque su significado no esté representado mediante campos explícitos.

Datos heterogéneos

Los tres tipos pueden convivir



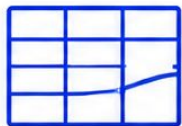
Una arquitectura IoT rara vez trabaja con un único tipo de dato.

6

Tecnologías de Almacenamiento

Distintos modelos para distintos problemas.

• ————— • Panorama de tecnologías de almacenamiento • ————— •



Relacionales

tablas y relaciones



NoSQL

estructuras flexibles



Series temporales

datos con tiempo



Especializadas

problemas específicos



Distintos modelos para distintos problemas.

Mapa de tecnologías

No existe una única forma de almacenar información

Podemos encontrar:

- bases relacionales;
- bases documentales;
- clave-valor;
- columnares;
- grafos;
- series temporales;
- almacenamiento geoespacial;
- almacenamiento en memoria;
- almacenamiento de objetos;
- otras tecnologías especializadas.



Relacionales



Documentales



Clave-valor



Columnares



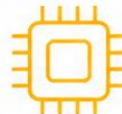
Grafos



Series
temporales



Geoespacial



En memoria



De objetos



Especializadas



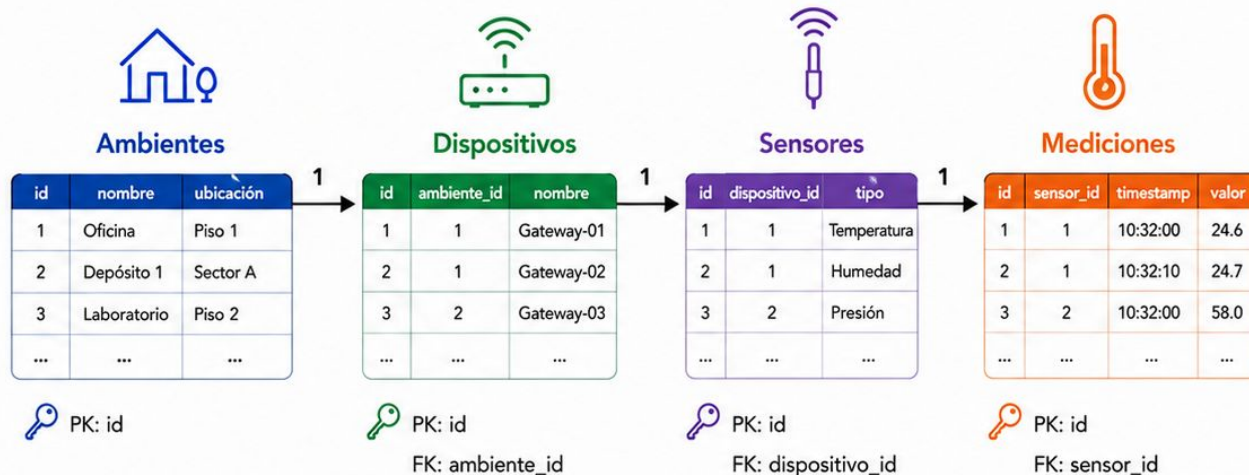
Cada modelo prioriza diferentes características.

Bases Relacionales

Datos estructurados y relaciones

Se caracterizan por:

- tablas;
- filas;
- columnas;
- claves;
- relaciones;
- reglas de integridad;
- lenguaje SQL.



Ejemplos



PostgreSQL



MySQL

ORACLE

Oracle



SQL Server



Las bases relacionales son ideales cuando los datos tienen estructura fija y las relaciones son importantes.

Bases Documentales

Información representada mediante documentos

Permiten almacenar estructuras flexibles y anidadas.

Pueden resultar útiles cuando:

- los atributos varían;
- la estructura cambia frecuentemente;
- existe información jerárquica.



Ejemplo
MongoDB



Documento 1

```
{
  "device_id": "dev-001",
  "ambiente": "laboratorio",
  "sensores": {
    "temperatura": 23.5,
    "humedad": 45
  },
  "ubicación": {
    "edificio": "A",
    "piso": 2,
    "sala": "A-201"
  },
  "estado": "activo"
}
```

Documento 2

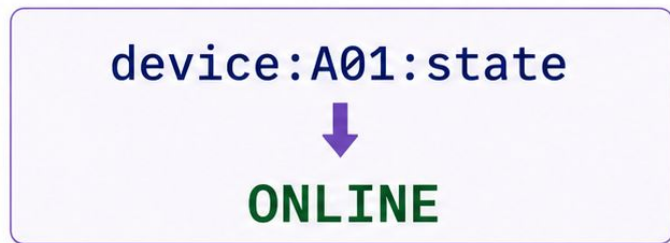
```
{
  "device_id": "dev-002",
  "ambiente": "exterior",
  "sensores": [
    {
      "tipo": "temperatura",
      "valor": 18.2,
      "unidad": "°C"
    },
    {
      "tipo": "humedad",
      "valor": 72,
      "unidad": "%"
    }
  ],
  "bateria": {
    "nivel": 88,
    "ultimo_cambio": "2025-05-18"
  }
}
```



Las estudiaremos con mayor profundidad cuando veamos **NoSQL**.

Bases Clave-Valor

Acceso rápido mediante una clave

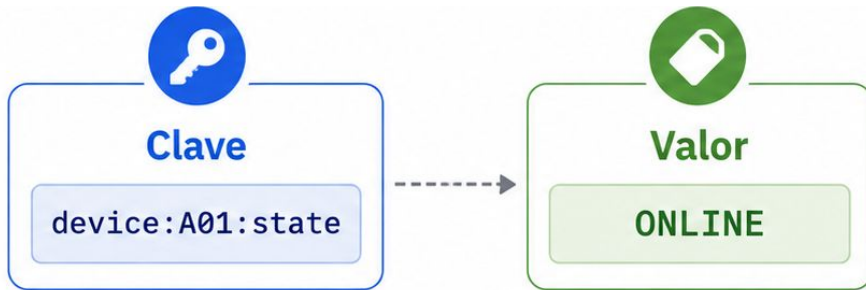


Son útiles para:

- caché;
- sesiones;
- estados temporales;
- valores de acceso frecuente.



Ejemplo
Redis



Acceso rápido

Lectura en
tiempo constante



Caché

Reduce latencia y
carga en sistemas



Estados temporales

Datos con
caducidad o TTL



Una **clave** permite recuperar: rápidamente un **valor** asociado.

Bases Columnares Distribuidas

Pensadas para distribución y escala

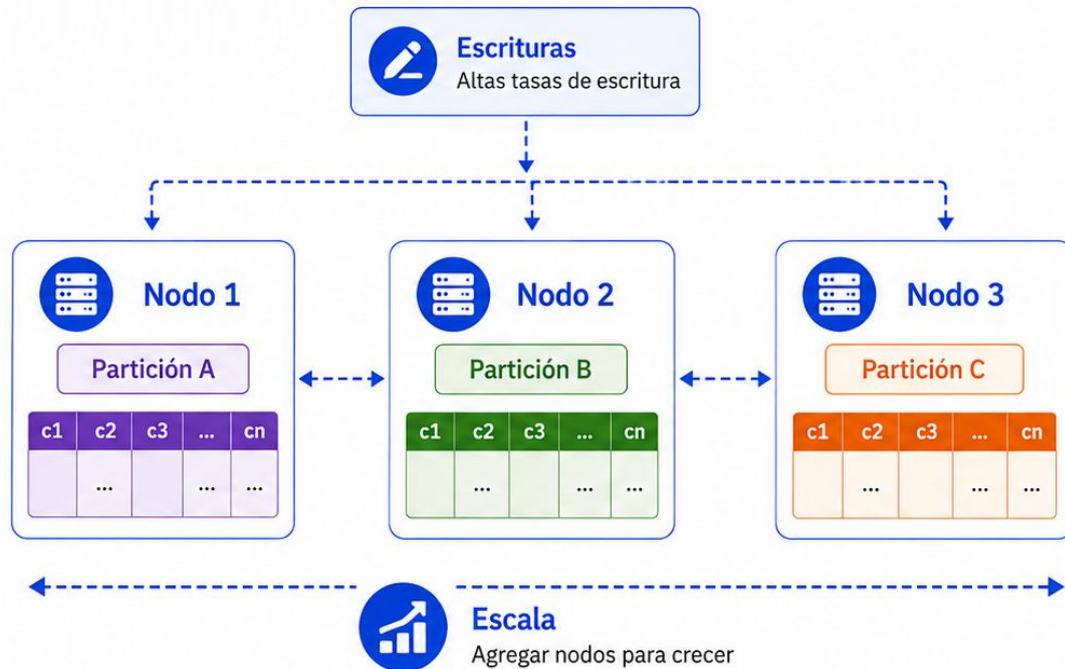
Pueden utilizarse cuando existen:

- grandes volúmenes;
- muchas escrituras;
- distribución entre nodos;
- patrones de consulta conocidos.



Ejemplos

Cassandra
ScyllaDB



El modelo de acceso influye fuertemente en el diseño.

Bases de Grafos

Subtítulo

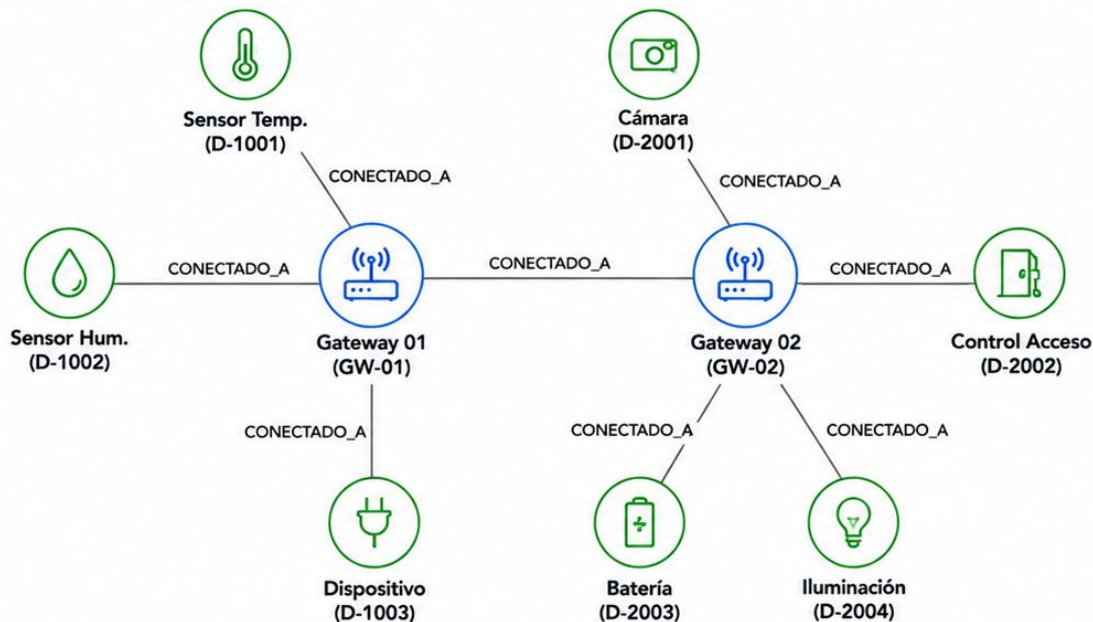
Representan:

- nodos;
- relaciones;
- propiedades.

En IoT pueden utilizarse para representar:

- dispositivos;
- gateways;
- dependencias;
- redes;
- topologías.

Ejemplo

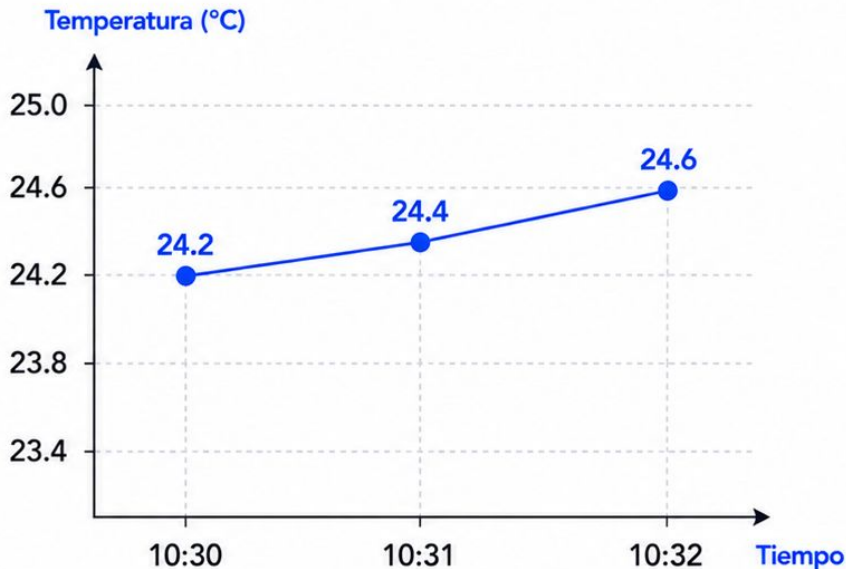


Los grafos permiten explorar conexiones complejas de manera eficiente.

Bases de Series Temporales

Cuando el tiempo es parte central del dato

Timestamp + dispositivo + variable + valor			
 10:30		Temperatura	24.2 °C
 10:31		Temperatura	24.4 °C
 10:32		Temperatura	24.6 °C



Ejemplos



TimescaleDB



InfluxDB



La telemetría IoT suele producir naturalmente series temporales.

Información geoespacial

Cuando necesitamos conocer dónde están las cosas

Permite responder preguntas como:

- ¿qué dispositivos están cerca?;
- ¿cuáles se encuentran dentro de una región?;
- ¿qué distancia existe entre dos ubicaciones?;
- ¿qué dispositivos recorrieron determinada zona?

Ejemplo



PostGIS



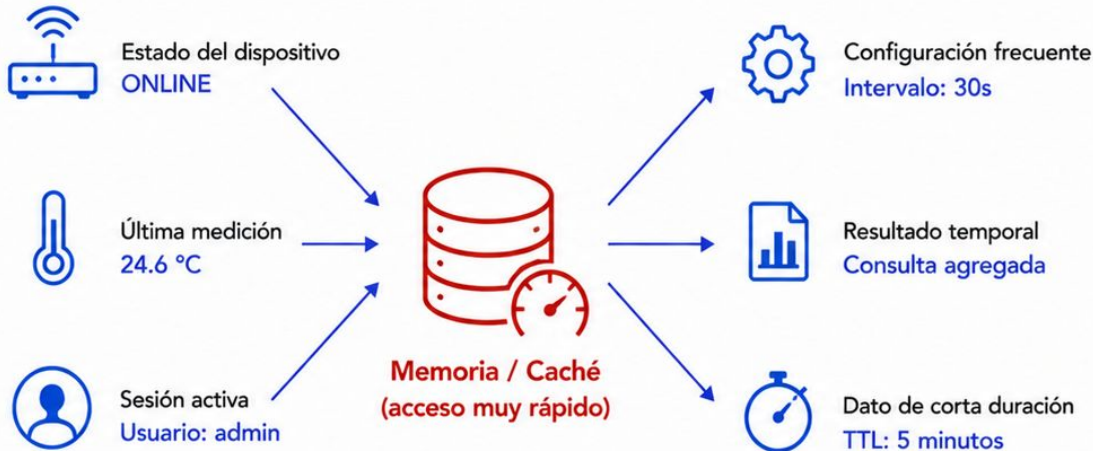
Los datos geoespaciales permiten analizar ubicación, proximidad y recorridos.

Almacenamiento en memoria

Datos que necesitamos recuperar rápidamente

Puede utilizarse para:

- último estado del dispositivo;
- última medición;
- sesiones;
- configuraciones frecuentes;
- resultados temporales;
- datos de corta duración.



No todo dato necesita la misma duración ni el mismo mecanismo de acceso.

Almacenamiento de objetos

Cuando el dato principal es un archivo

Puede almacenar:

- fotografías;
- video;
- audio;
- firmware;
- backups;
- archivos generados por dispositivos.

Ejemplos



Amazon S3



MinIO

Base de datos



→ metadatos

Object Storage



→ contenido



Muchas arquitecturas separan los archivos de la información que los describe.

Tecnologías especializadas

Nuevos problemas generan nuevas herramientas

También existen tecnologías especializadas para:

- búsqueda de texto;
- almacenamiento vectorial;
- analítica;
- procesamiento distribuido;
- eventos y flujos continuos.



Búsqueda
de texto



Almacenamiento
vectorial



Analítica



Procesamiento
distribuido



Eventos y
flujos continuos



Hoy necesitamos reconocer el panorama.
Las tecnologías relevantes se profundizarán durante la materia.

Persistencia políglota

Combinar tecnologías según la necesidad



PostgreSQL



dispositivos, ambientes
y configuraciones



Series temporales



mediciones



Redis



último estado o
última medición



Una arquitectura puede utilizar más de un mecanismo de almacenamiento.

Crterios de seleccin

La tecnologa aparece despus del requerimiento

Debemos considerar:

- estructura;
- volumen;
- frecuencia de generacin;
- patrn de consulta;
- consistencia;
- latencia;
- retencin;
- disponibilidad;
- escalabilidad;
- costo.



estructura



volumen



frecuencia de
generacin



patr3n de
consulta



consistencia



latencia



retencin



disponibilidad



escalabilidad



costo



La pregunta no es cul tecnologa es mejor.
La pregunta es cul responde mejor a este problema.

7

Persistencia y Ciclo de Vida

Los datos también tienen una historia.



Los datos también tienen una historia.

Ciclo de vida de los datos

Desde que se generan hasta que dejan de ser necesarios



Diseñar una arquitectura de datos también implica pensar qué ocurre con la información a lo largo del tiempo.

Persistencia

Guardar información también es una decisión

No todos los datos necesitan conservarse de la misma manera.

Debemos preguntarnos:

- ¿necesitamos volver a consultarlos?;
- ¿durante cuánto tiempo?;
- ¿tienen valor histórico?;
- ¿pueden reconstruirse?;
- ¿podemos conservar solamente parte de la información?



Persistir no significa guardar todo para siempre.

Valor de los datos

Las necesidades pueden cambiar con el tiempo



Datos recientes



mayor nivel de detalle



Datos históricos



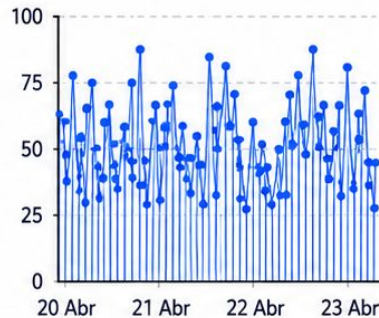
pueden resumirse
o agregarse

Por ejemplo, podemos necesitar:

- mediciones **detalladas** para conocer lo ocurrido recientemente;
- información **resumida** para estudiar períodos largos.

**Datos recientes
detallados**

*(muchas mediciones
individuales)*

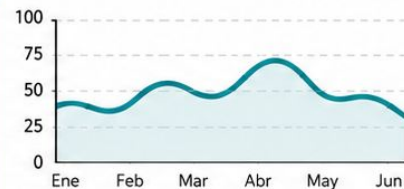


Nivel de detalle: alto
Horizonte temporal: corto



**Datos históricos
resumidos**

*(información agregada
en el tiempo)*



Promedio
52



Máximo
78



Mínimo
28

Nivel de detalle: bajo
Horizonte temporal: largo



El nivel de detalle necesario depende del uso que hagamos de la información.

Más que mediciones

El dato necesita contexto para conservar su significado

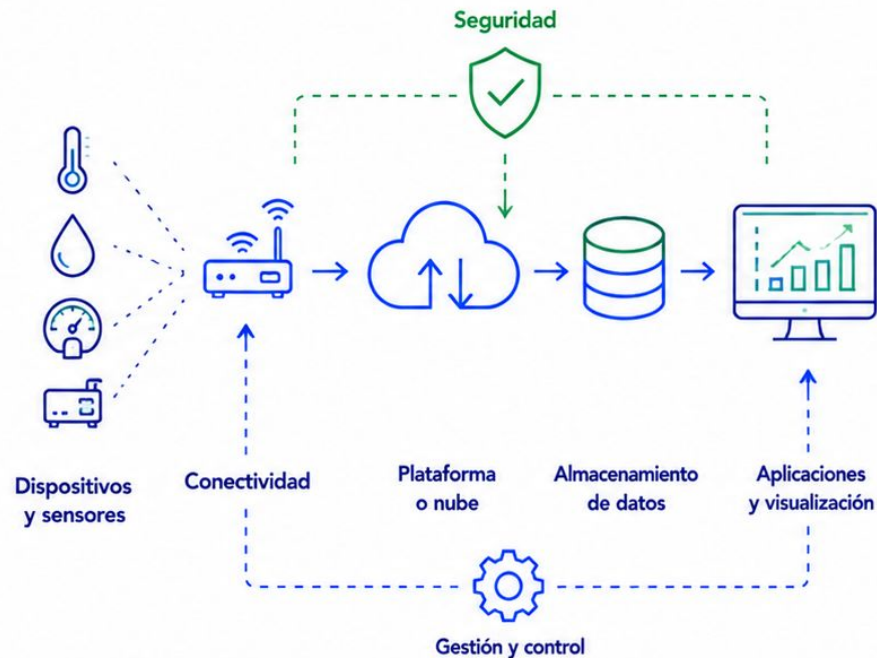


Una medición aislada tiene poco valor si perdemos el contexto que permite interpretarla.

8

Una Arquitectura IoT Completa

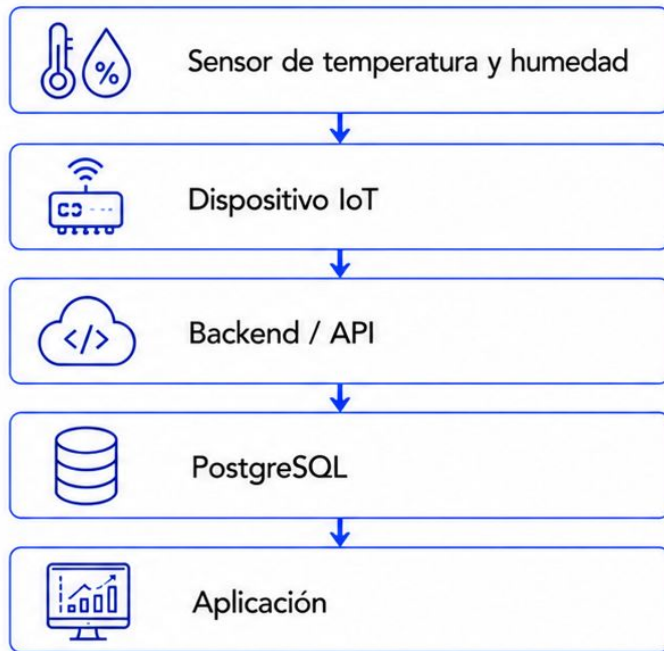
Ahora podemos reunir las piezas.



Cada componente cumple un rol. El valor surge cuando todos se integran.

Una primera arquitectura

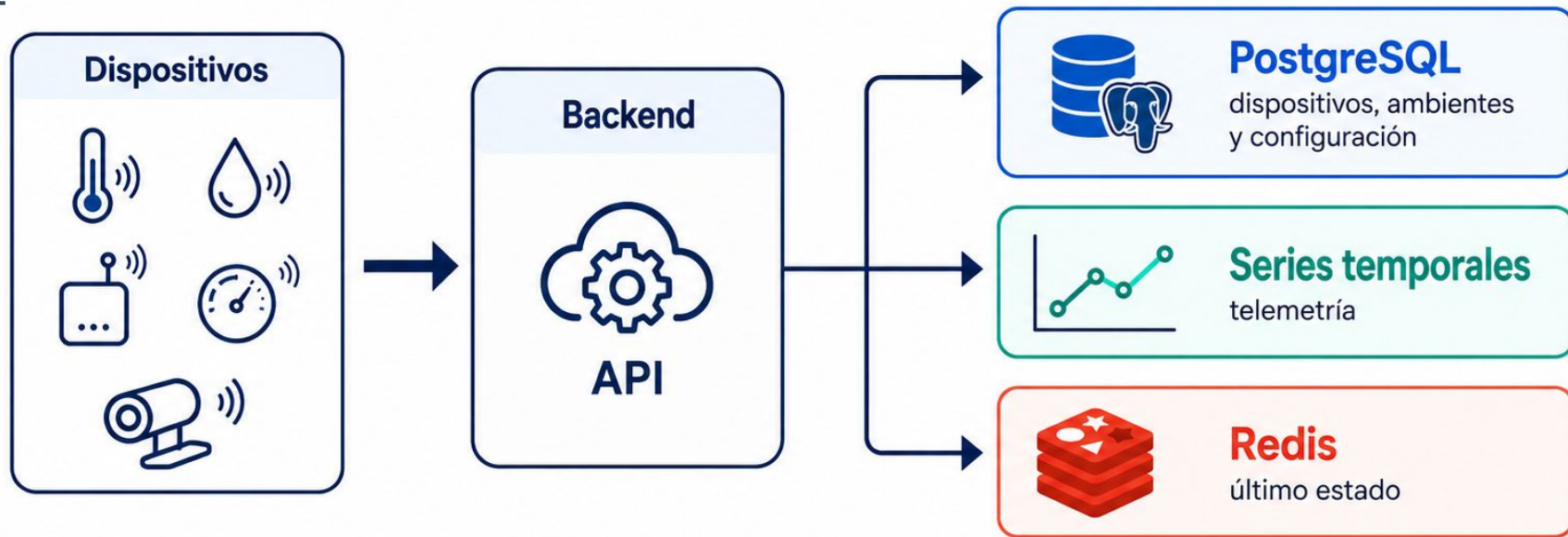
Empezar por lo necesario



Una buena arquitectura no necesita comenzar siendo compleja.

Cuando hay nuevas necesidades

Distintos requerimientos pueden incorporar nuevos mecanismos



La complejidad debe aparecer como respuesta a un requerimiento.

Mirar IoT desde los datos

Una perspectiva diferente de la arquitectura



La arquitectura de datos describe el recorrido completo de la información.

9

Caso de Análisis

Monitoreo de ambientes.



Monitoreo de ambientes.

Monitoreo de ambientes

Un sistema IoT sencillo

Un dispositivo instalado en un ambiente registra periódicamente:

- temperatura;
- humedad relativa;
- presión atmosférica, si el sensor la proporciona.

Las mediciones se envían a una aplicación para consultar el estado del ambiente y su evolución.



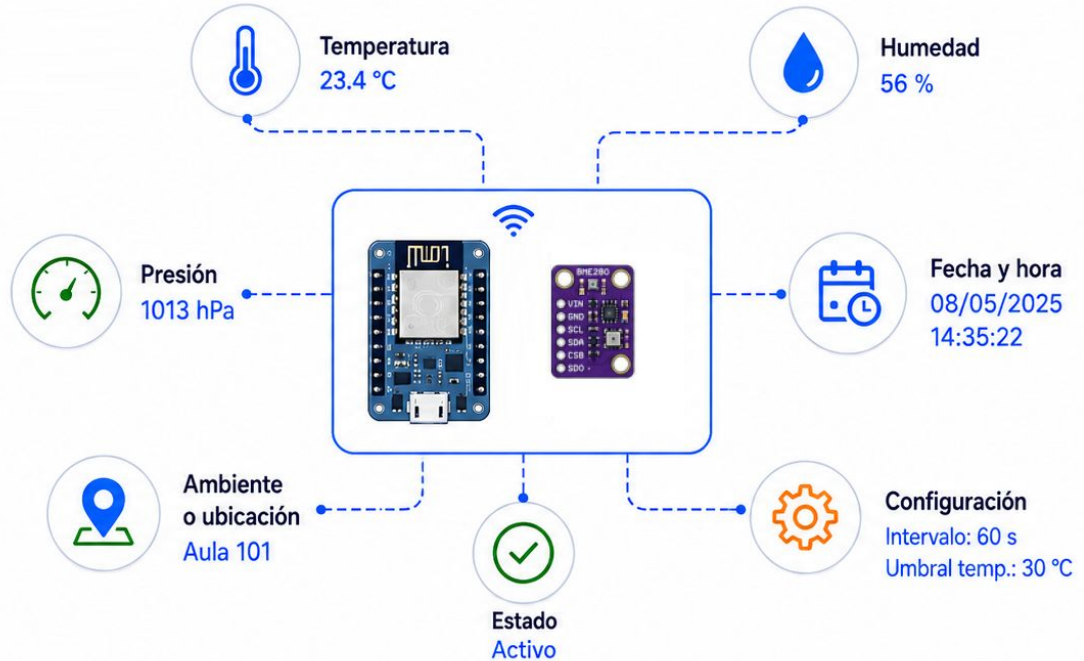
Vamos a utilizar un caso simple para identificar las decisiones básicas de una arquitectura de datos.

¿Qué datos existen?

Primera pregunta

Podemos encontrar:

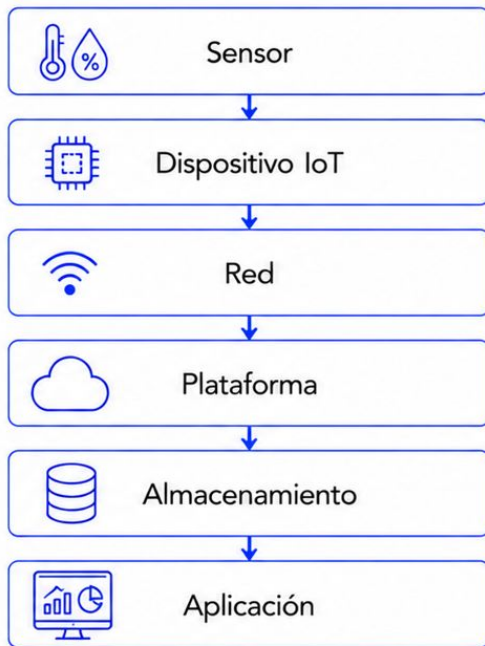
- dispositivo;
- sensor;
- ambiente o ubicación;
- temperatura;
- humedad;
- presión;
- fecha y hora;
- estado;
- configuración.



Antes de pensar dónde almacenar los datos, debemos entender qué información existe.

¿Cómo circulan los datos?

Segunda pregunta



La arquitectura de datos comienza siguiendo el recorrido de la información.

¿Qué necesitamos conservar?

Tercera pregunta

Podemos necesitar:



Estado actual



última temperatura;



última humedad;



última presión;



estado del dispositivo.



Histórico



evolución de las mediciones;



cambios relevantes;



información necesaria para
análisis posteriores.



No todos los datos tienen que conservarse de la misma manera ni durante el mismo tiempo.



Para ir cerrando..

Arquitectura antes que tecnología

Entender el problema antes de elegir herramientas

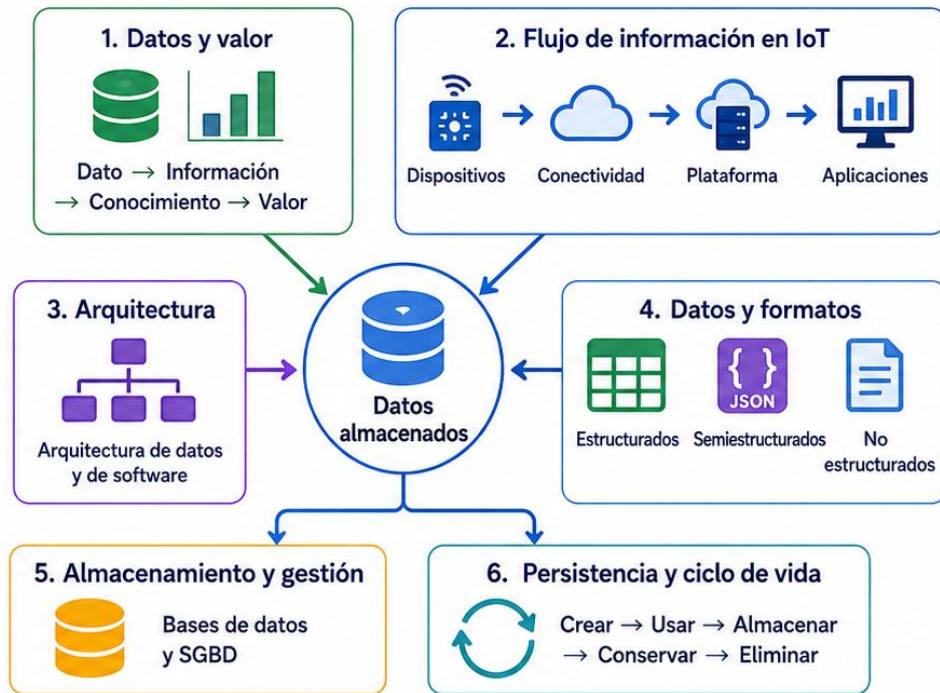


PostgreSQL, MongoDB, Redis o cualquier otro motor son herramientas.
La arquitectura surge de los requerimientos.

Resumen de la clase

Ideas principales

- Dato, información y conocimiento.
- Arquitectura de datos y arquitectura de software.
- Flujo de información en IoT.
- Fuentes, ingreso, almacenamiento, procesamiento y consumo.
- Evolución de las tecnologías de datos.
- Bases de datos y SGBD.
- Datos estructurados, semiestructurados y no estructurados.
- Familias de tecnologías de almacenamiento.
- Persistencia y ciclo de vida.



La arquitectura de datos define cómo la información nace, circula, persiste y finalmente se utiliza.

El mapa de la materia

Lo que profundizaremos



Hoy vimos el mapa.

Durante las próximas clases vamos a recorrerlo.

Próxima clase

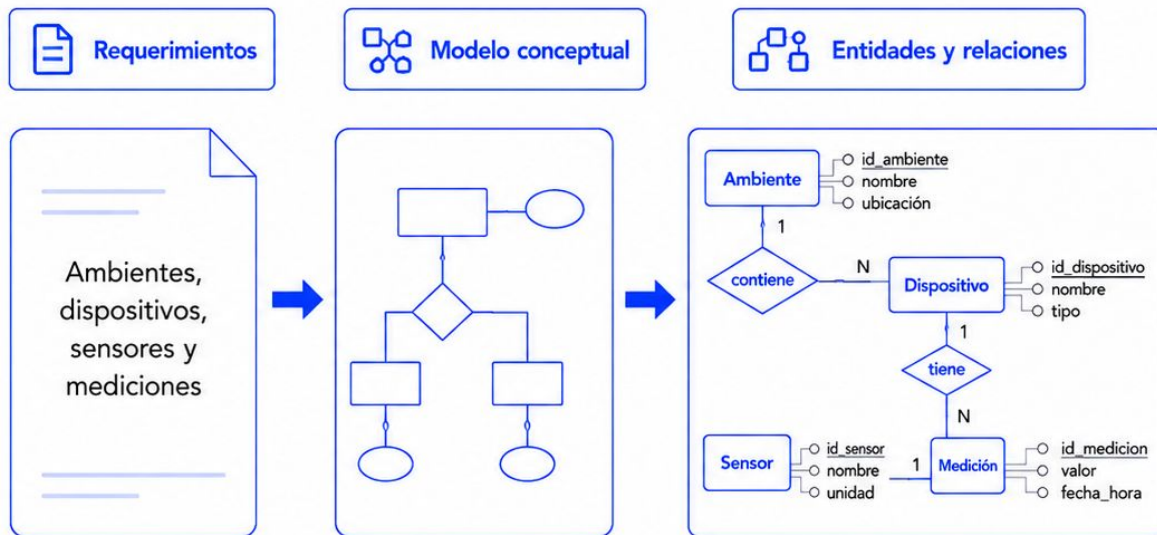
Subtítulo

Vamos a transformar requerimientos como:

Necesitamos almacenar ambientes, dispositivos, sensores y mediciones.

en estructuras formales mediante:

- entidades;
- atributos;
- relaciones;
- identificadores;
- cardinalidades;
- restricciones;
- estados e historiales.



**Hoy entendimos dónde viven y cómo circulan los datos.
La próxima clase veremos cómo representarlos.**



Muchas gracias ¿Dudas?

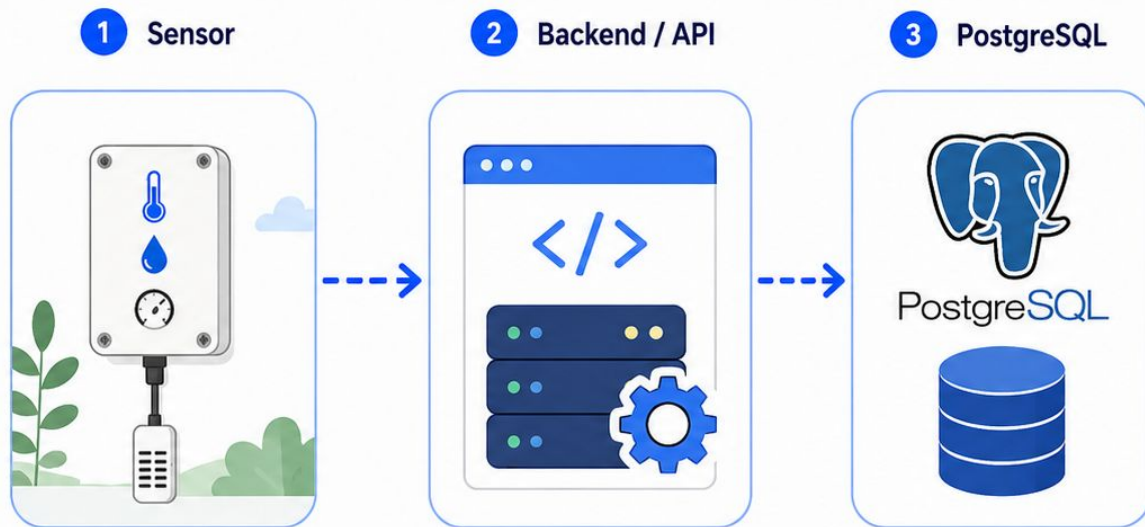


Enlace a la encuesta

10

Práctica

Persistir y recuperar datos de un sistema IoT.



Persistir y recuperar datos de un sistema IoT.

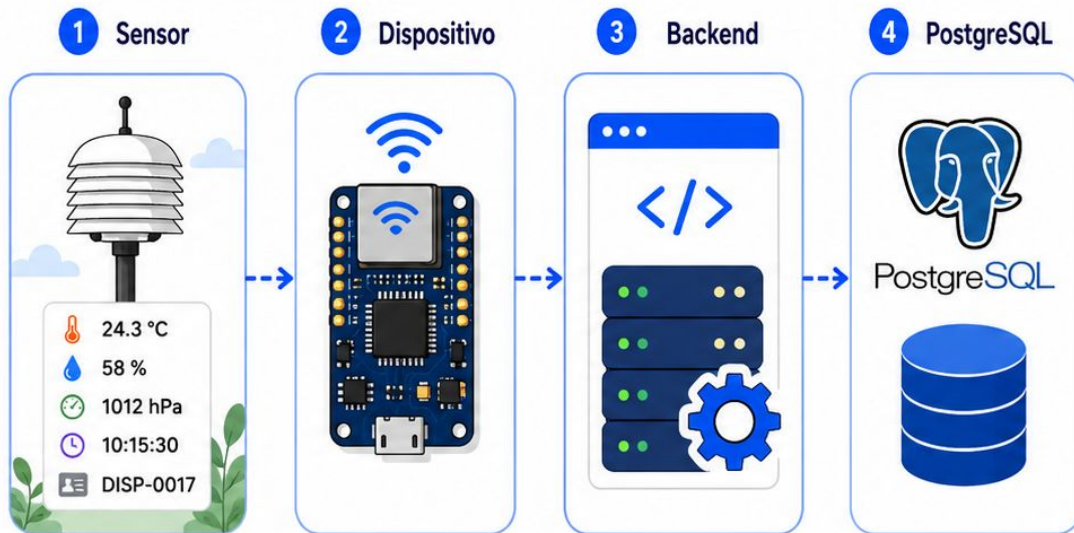
Monitoreo de ambientes

Del sensor a la base de datos

Vamos a utilizar un dispositivo simulado que genera periódicamente:

- temperatura;
- humedad relativa;
- presión atmosférica, cuando esté disponible;
- fecha y hora;
- identificador del dispositivo.

Las mediciones serán recibidas por una aplicación y almacenadas en PostgreSQL.



Vamos a observar cómo un dato generado por un dispositivo se transforma en información persistida.

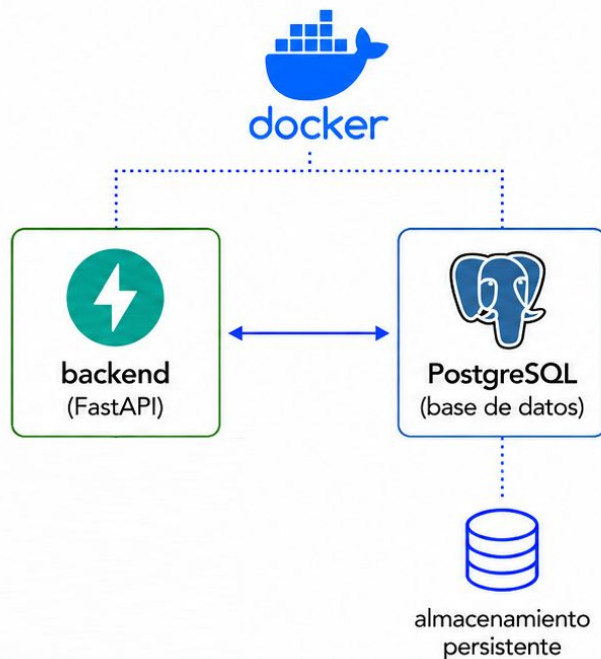
Paso 1

Levantar PostgreSQL y la aplicación

```
$ cd clase-01  
$ cp .env.example .env  
$ docker compose up -d  
$ docker compose ps
```

Vamos a identificar:

- backend;
- PostgreSQL;
- conexión entre ambos;
- almacenamiento persistente.



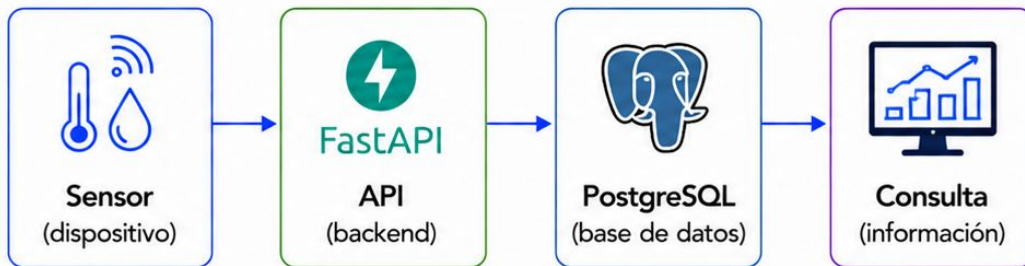
**PostgreSQL será el SGBD
encargado de almacenar las mediciones.**

Paso 2

Registrar mediciones y recuperar información

Vamos a:

- enviar una medición manual;
- generar mediciones con el simulador;
- consultar las mediciones almacenadas;
- recuperar la última medición;
- comparar estado actual e histórico.



Persistir permite recuperar un dato después de que fue generado.

Paso 3

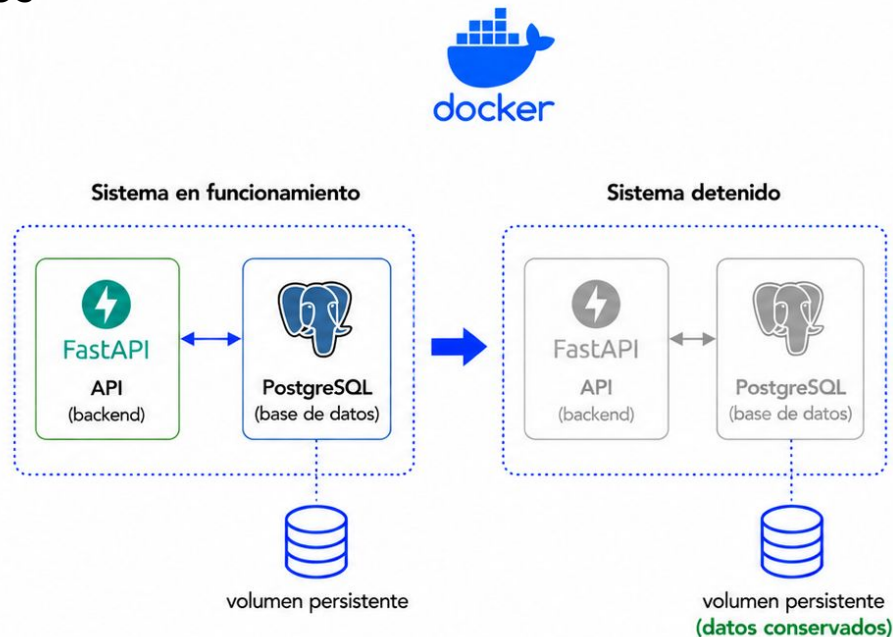
Detener el sistema sin perder los datos

```
$ docker compose down  
$ docker compose up -d
```

Después volveremos a consultar las mediciones.

Finalmente compararemos con:

```
$ docker compose down -v
```



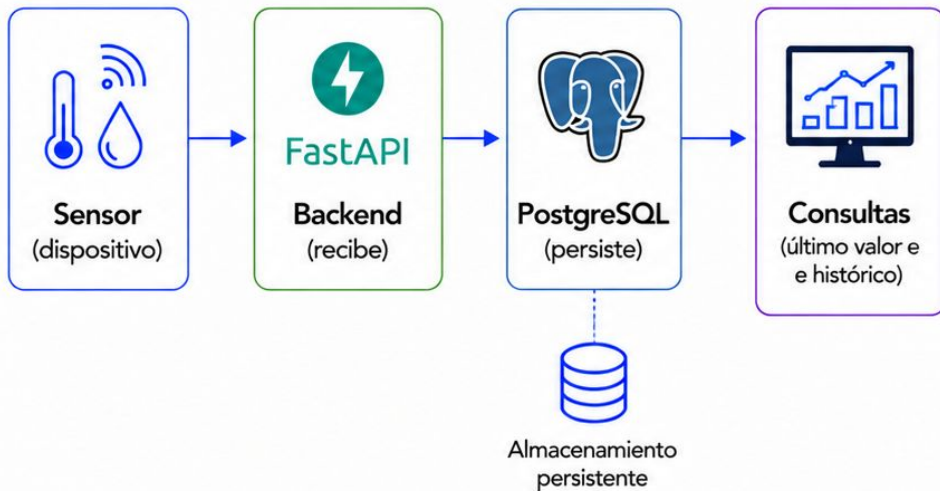
**Detener una aplicación no implica necesariamente perder los datos.
Eliminar el almacenamiento persistente sí.**

¿Qué observamos?

Relacionar la práctica con los conceptos

Vamos a identificar:

- qué datos genera el dispositivo;
- qué información acompaña cada medición;
- qué componente recibe los datos;
- qué componente los persiste;
- qué diferencia existe entre último valor e histórico;
- qué limitaciones tiene nuestra estructura inicial.



Hoy observamos cómo persisten los datos.

En la próxima clase veremos cómo modelarlos correctamente.

Enlace al repositorio GitHub